Simulation of Domain Expert in LLM models

A Project Report
Presented to
The Faculty of the Department of Applied Data Science
San Jose State University
In Partial Fulfillment
Of the Requirements for the Degree
Master of Science in Applied Data Intelligence

By

Saqib Chowdhury
Darpankumar Jiyani
Prathyusha Pingili
Manjot Singh
Sai Prasad Thalluri

December 5, 2025

Copyright © 2025
Saqib Chowdhury, Darpankumar Jiyani, Prathyusha Pingili, Manjot Singh, Sai Prasad Thalluri
ALL RIGHTS RESERVED

APPROVED FOR DEPARTMENT OF APPLIED DATA SCIENCE

Dr. Simon Shim, Project Advisor

Dr. Lee C. Chang, Project Advisor

ABSTRACT

Simulation of Domain Expert in LLM models

By

Saqib Chowdhury, Darpankumar Jiyani, Prathyusha Pingili, Manjot Singh, Sai Prasad Thalluri

The development of a specialized large language model (LLM) for data science and mathematics, programming, and theoretical reasoning tasks is described in this document. The project created an end-to-end pipeline for preparing the data set (curation, preprocessing, benchmark evaluation), selecting the best model based on benchmarks, fine-tuning the QLoRA (Qualitative and Longitudinal Reassurance Assessment) process on these models, and deploying them within a multi-model architecture. The curated data sets were converted into a standard instruction/response format that retained mathematical symbols/notation, code structures, and conceptual integrity so as to allow efficient and stable fine-tuning of each of the five resulting specialist models. A quantitative evaluation using the Global Scale Model 8K (GSM8K), MATH, Human Evaluate (HE), DS-1000, and BERTScore-F1 metrics showed significant increases in the accuracy of results, quality of reasoning shown in those results, and structure/reliability of those results versus a zero-shot baseline. The final system was deployed with a FastAPI back end, a React front end, and router to direct queries from users automatically to the most appropriate specialist model. A comprehensive safety layer was developed for safe and appropriate use of the system. The results demonstrate how targeted pre-processing, structured evaluations, and modular deployments of these LLMs create high-quality data science experts that are useful for educational and research purposes and for professional applications.

ACKNOWLEDGEMENT

We would like to thank Dr. Simon Shim and Dr. Lee C. Chang, our project advisor for the guidance and support throughout the semester. Their experience and expertise in Artificial Intelligence, Systems Design and Research Methodologies were helpful in both the direction of this project and the overall quality of the project.

We also wish to thank San Jose State University for supplying the resources, academic atmosphere, and infrastructure which enabled us to complete this project. In addition, we wish to give our heartfelt thanks to the open-source communities that contributed to the tools and models used in this project, (including QLoRA, Unsloth, Hugging Face, FastAPI, React, and OpenRouter), whose efforts made it possible for us to conduct and deploy large-scale experimentation and research.

Last but not least, we thank all our classmates, peers, friends, and family for the encouragement, support, and collaboration we received throughout this project experience. Their support and motivation made the completion of our project possible.

TABLE OF CONTENTS

Chapter 1 Introduction
- **1.1 Project Background and Execute Summary**
- **1.2 Project Requirements**
- **1.3 Project Deliverables**
- **1.4 Technology and Solution Survey**
- **1.5 Literature Survey of Existing Research**

Chapter 2 Data and Project Management Plan
- **2.1 Data Management Plan**
- **2.2 Project Development Methodology**
- **2.3 Project Organization Plan**
- **2.4 Project Resource Requirements and Plan**
- **2.5 Project Schedule**

Chapter 3 Data Engineering
- **3.1 Data Process**
- **3.2 Data Collection**
- **3.3 Data Pre-processing**
- **3.4 Data Transformation**
- **3.5 Data Preparation**
- **3.6 Data Statistics**
- **3.7 Data Analytics Results**

Chapter 4 Model Development
- **4.1 Model Proposals**
- **4.2 Model Supports**
- **4.3 Model Comparison and Justification**
- **4.4 Model Evaluation Methods**
- **4.5 Model Validation and Evaluation Results**

Chapter 5 Data Analytics System
- **5.1 System Requirements Analysis**
- **5.2 System Design**
- **5.3 Intelligent Solution**
- **5.4 System Development and Implementation**

Chapter 6 System Evaluation and Visualization

- **6.1 Analysis of Model Execution and Evaluation Results**
- **6.2 Achievements and Constraints**
- **6.3 Quality Evaluation of Model Functions and Performance**
- **6.4 Evaluation of Models Vs Requirements**
- **6.5 Project Information Visualization**

Chapter 7 Conclusion

- **7.1 Summary**
- **7.2 Benefits and Shortcoming**
- **7.3 Potential System and Model Applications**
- **7.4 Experience and Lessons Learned**
- **7.5 Recommendations for Future Work**
- **7.6 Contributions and Impacts on Society**

References

Appendices

Appendix A – System Testing

Appendix B – Project Data Source and Management Store

Appendix C –Project Program Source Library, Presentation and Demonstration

Chapter 1 Introduction

1.1 Project Background and Executive Summary

LLM, or Large Language Models, are now critical components of Data Science. They solve complex problems in Data Science like; Mathematical reasoning, Code generation, Statistical explanation, and Conceptual Analysis. The generation of general capabilities from frontier models (e.g. GPT-4, Gemini) currently has strong general capabilities across domains but lacks consistency in some Domain-Specific Use Cases due to the absence of targeted training datasets, domain-specific reasoning frameworks and controlled evaluation methods. The purpose of the project is to create a Domain Expert LLM to fill this gap within Data Science Practice by focusing on 3 areas: Domain-Specific Expert Theoretical Knowledge; Domain-Specific Mathematical Problem Solving Skills; Domain-Specific Programming Skills.

The overall goal of the project was to develop a single end-to-end system for building quality data sets; performing thorough preprocessing of data; evaluating many of the open source versions of "ranking" an assortment of software models; and then using fine-tuning methods to create expert versions of the best-performing models. The three areas on which the research was performed—Mathematical, Coding and Classification Theory—each place strict constraints on the type of preprocessing techniques, standards for evaluation, and methods used for developing the architecture of the software. For example, the mathematical-related data sets must retain the integrity of both the LaTeX expressions, as well as the multi-step solution method for presenting the information. The coding-related data sets' indentation and hierarchy structures must also be preserved to prevent syntax corruption. The coding and classification theory-related data sets also require the normalisation of the free-form responses, removal of the placeholders, and converting them into a standardised instruction format. All three areas used a unified

JSONL conversational format structure. This format makes it easier to work with the most widely used machine learning training systems today.

To select the five models used for further work, a structured evaluation process was created. About 15 popular open source LLMs were compared on benchmark tasks including GSM8K, MATH, DS-1000. To ensure proper evaluation, the benchmark datasets and tasks were kept out of the training pipelines. Zero-shot results were compared using quantitative metrics and qualitative review by an LLM judge. The strongest models in their respective domain types were chosen for fine-tuning: Falcon-H1 and Qwen2.5-7B as the strongest theory models; AceMath-7B and Qwen2.5-Math as the strongest maths models; and Seed-Coder-8B as the strongest code model.

The project was completed on GCP, which allows for a scalable, repeatable operational platform. All raw and processed data were stored in designated Cloud Storage buckets (containers). The fine-tuned models used LoRA with the Transformers and Unsloth libraries; therefore, training runs were tracked through Weights and Biases. After training was completed for all three domains, evaluation data indicated an increased consistency in reasoning, increased accuracy, and improved quality of the output.

The completed system is being deployed in GCP, using FastAPI for backend and hosting with Cloud run, adding a safety layer using Llama Guard as well. The final application will allow users to send prompts to the best model for their needs via a routing mechanism which supports a "Mixture of Endpoints". Overall, this report represents a complete end-to-end methodology to build industry-leading LLM systems from the initial data collection stage to when they can be utilized by users. This report has shown that open-source models can achieve expert level

performance in a niche domain by utilizing good dataset design, a strong preprocessing pipeline, thorough benchmarking, and tightly controlled fine-tuning. In conclusion, this completed system provides a robust and modular foundation for future developments in the area of advanced data science applications and provides the foundation for future improvements such as Reinforcement Learning from Human Feedback, larger scale training, and the ability to deploy LLMs in wider domains.

1.2 Project Requirements

In order to develop a domain-expert language model system, it is essential to define and create a clear set of technical, functional, and architectural requirements that will result in a reliable, scalable, and accurate model. The original project was structured to meet three main requirements: (1) providing high-quality domain-specific training datasets; (2) offering a solid infrastructure that allows for preprocessing, fine-tuning, and evaluating the model; and (3) building an architecture that is ready to produce the final output from the model and also deliver that output efficiently and safely to end users.

To meet these requirements, the system must have datasets that have been carefully curated from three different areas of research: theory, mathematics, and code. All datasets must be processed to remove any duplication and then cleaned, normalized, and validated for their structural integrity. For example, all math data must be accessible in LaTeX or other reasonably standardized forms. Each code dataset must strictly maintain all associated indentation and syntax.

In addition to a robust dataset of training and evaluation materials, the system must also provide access to computational resources capable of performing parameter-efficient fine-tuning on these

datasets. This will require high-performance computational resources capable of performing pre-processing at a very large scale, running GPU-accelerated training jobs, and managing storage and checkpointing across different cloud-based computer systems. As such, this project will use many of the services offered by Google Cloud Platforms.

The third requirement for this system was to have a deployment framework for the exposed fine-tuned model through a cohesive interface. In order to meet this criteria, we specified all of our necessary elements for building a FastAPI backend, and utilizing a safety feature (Llama Guard) to restrict unsafe input and output.

1.3 Project Deliverables

The project generated a complete collection of deliverables, representing all phases of a dataset creation, model development, and system deployment. The deliverables include:

- **Curated and Pre-Processed Datasets**
    - Unified JSONL files for theory, math, code
    - LaTex, Symbolic, and Code structure retention verified
    - Cleaned datasets
- **Benchmarking Reports and Visualizations**
    - Zero-shot evaluation of 15+ candidate models
    - Domain-specific charts for each model relative to performance (Math, code, Theory)
    - Baseline and Fine-Tuned Scores comparison

- **Fine-Tuned Specialist Models**
    - Five domain-expert models:

      Falcon-H1-7B (Theory), Qwen2.5-7B (Theory), AceMath-7B (Math),

      Qwen2.5-Math-7B (Math), Seed-Coder-8B (Code)
    - Trained using LoRA
    - Complete WanDB Logs for Traceability
- **Evaluation Framework and Results**
    - Evaluation of GPT-as-a-Judge using qualitative assessment
    - Benchmark and Quantitative Method used to evaluate models
    - Reasoning for Selecting Models
- **Deployment Architecture**
    - FastAPI backend hosted on cloud run
    - Safety and routing mechanisms
- **Final Project Report and Documentation**
    - Technical report in APA format
    - System diagrams, methodology, and analysis
    - Issues/Records of Preprocessing, Training, Evaluating and Deploying Workflows

These deliverables represent a comprehensive view of the technical pipeline from RAW data input to inference performed using a service hosted in the cloud.

1.4 Technology and Solution Survey

This project involved using multiple types of technology to develop an overall solution that includes Data Engineering, Machine Learning (ML), and Training Optimization, in addition to deployment to a Cloud environment; therefore, many current Technology Products, Development Frameworks, and Service Providers, will support building a scalable LLM Solution.

From a Training perspective, both the Transformers library and the Unsloth library were reviewed to determine which of the two products were best suitable to support Parameter-Efficient Fine-Tuning (PEFT) of 7B-9B Models. Unsloth was chosen as the preferred library based on its efficient use of LoRA implementations that help to lower the required memory for Training Models on A100, H100 GPUs. PEFT provided the core LoRA Functionality to facilitate the desired level of PEFT.

A number of industry-standard Tools were reviewed for Evaluation purposes. The lm-evaluation-harness toolset was used for benchmarking the Overall LLM Capability, using the following benchmark datasets: GSM8K, MATH, DS-1000. In generating the Qualitative Evaluation of the Models, GPT-4o was chosen because of its high accuracy and consistency in terms of Reasoning and Grading Stability.

In terms of Infrastructure, a number of Google Cloud Platform (GCP) Services were examined for a number of different things (Compute, Storage, Deployment, etc.). Cloud Storage provides a centralised storage location for Data. Cloud Run allows the FastAPI Backend to be delivered in a lightweight, Serverless manner. The technology stack ultimately created a framework that allows

for an efficient, reproducible, and scalable manner to build LLM technology for domain professionals.

1.5 Literature Survey of Existing Research

Large language models (LLMs) have moved from generic text generators to central components in domain-specific assistants for medicine, law, finance, and the sciences. Ling et al. argue that this shift is driven by domain specialization: shaping models, data, and objectives around a specific domain instead of relying on purely general-purpose behavior. They categorize specialization methods into black-box (e.g., retrieval-augmented generation and tool calling), gray-box (prompt templates and soft prompts), and white-box (fine-tuning, adapters, and continual pretraining), and show that successful domain-specific LLMs are built on targeted domain corpora, explicit domain knowledge, and domain-aligned evaluation. Wang et al., in a survey of factuality, similarly highlight "domain-specific factual LLMs" for healthcare, finance, education, food testing, and geoscience, emphasizing that these models either embed domain knowledge in their parameters or access it via retrieval over trusted resources.

Practical guides from industry mirror this picture. The SuperAnnotate report on domain-specific LLMs describes how organizations curate in-domain data, annotate it, fine-tune a base LLM, and then continuously monitor domain-relevant metrics; the focus is on models that "speak the language" of a vertical and respect its constraints. The PromptEngineering.org guide explains how to inject domain expertise via both prompting (domain instructions, in-context examples, tool use, retrieval) and parameter updates (instruction tuning, continued pretraining, LoRA-style adapters), stressing that evaluation must be domain-specific even if only prompts are changed. In healthcare, for example, Wang et al. describe models such as HuatuoGPT and Zhongjing that are

trained or continually tuned on clinical notes and medical QA corpora and evaluated on medical benchmarks like cMedQA2, with explicit attention to guideline consistency and safety.In finance, Jeong outlines how a base model such as Falcon-7B can be adapted with parameter-efficient fine-tuning on curated financial QA datasets to outperform generic LLMs on specialized questions.

In addition to applications that deal only with text, there has been recent interest in applying LLMs in science and technical fields using LLMs-DSK (Domain-Specific Knowledge) for ocean environment prediction. By creating a hybrid model of LLMs and SD knowledge, researchers can use oceanographic understanding as part of their prediction process; thus improving both the accuracy and interpretability of the prediction(s) (e.g., increases in correlation between prediction and observed/test results). Suh et al.show that LLMs can generate domain-targeted contextual descriptions that significantly improve domain-specific automatic speech recognition, sometimes even outperforming human-written descriptions. These examples illustrate a general pattern: domain-specific LLMs increasingly serve as intelligent interfaces to structured, domain-specific systems, rather than simply as text generators.

In addition to applications that deal only with text, there has been recent interest in applying LLMs in science and technical fields using LLMs-DSK (Domain-Specific Knowledge) for ocean environment prediction. By creating a hybrid model of LLMs and SD knowledge, researchers can use oceanographic understanding as part of their prediction process; thus improving both the accuracy and interpretability of the prediction(s) (e.g., increases in correlation between prediction and observed/test results). Suh et al.show that LLMs can generate domain-targeted contextual descriptions that significantly improve domain-specific automatic speech recognition, sometimes even outperforming human-written descriptions. These examples illustrate a general

pattern: domain-specific LLMs increasingly serve as intelligent interfaces to structured, domain-specific systems, rather than simply as text generators.

Subsequent benchmarks broaden this perspective. Kadiyala et al. propose DSBC (Data Science Benchmark with Context Engineering) to capture multi-label data-science tasks that span statistics, parsing, preprocessing, feature engineering, and distribution analysis, often requiring several operations in a single query over real data files. DSBC and related benchmarks such as DABStep, DataSciBench, DA-Code, DSBench, DSEval, and Arcade together define a rich evaluation space: from Excel-like operations and single-cell transformations to notebook-style, multi-cell pipelines and agentic decompositions. Fang et al. survey LLMs on tabular data and identify core capabilities needed for such tasks—understanding schema and semantics, reasoning over heterogeneous data types, and generating correct transformation logic and explanations—arguing that generic LLMs are not yet fully reliable on these fronts. Despite this ecosystem of benchmarks, most reported results still involve general-purpose models (GPT-4, CodeLlama, etc.) or lightly tuned variants, rather than a clearly defined, domain-specific LLM for data science.

In parallel, many authors have explored data science agents that wrap LLMs with tools and workflows. Hong et al. present Data Interpreter, an LLM-based agent that decomposes complex data-science tasks into subproblems using a hierarchical graph model with node types such as “Code,” “Execute,” “Analyze,” and “Understand.” Data Interpreter adapts its plan based on intermediate results and achieves strong performance on data-analysis and machine-learning benchmarks like InfiAgent-DABench, demonstrating that general LLMs can be scaffolded to perform sophisticated workflows. Guo et al's DS-Agent applies a case-based reasoning strategy to locate previously implemented data-science solutions for new tasks. They utilize an LLM to

reason about similarities between prior solutions with new tasks and to adapt prior solutions for new tasks. The CAAFE developed by Hollmann et al uses LLMs for context-based automated feature engineering, while Yang et al's MatPlotAgent applies LLMs to develop and improve upon scientific visualizations.Sun et al. and Wang et al. survey such LLM-based agents for statistics and data science, concluding that most rely on powerful general-purpose models combined with handcrafted workflows and tool calls, rather than on a unified data-science-specialized base model.

Other domain-specific systems provide useful templates without covering the full data-science lifecycle. Huang et al. propose DSQA-LLM, a domain-specific intelligent question-answering framework that combines a domain classifier, XLNet, and GPT-3.5-turbo to route and answer questions within a particular domain. DSQA-LLM illustrates how domain-aware routing and mixed-model pipelines can yield coherent answers in specialized settings. LLM-DSK, as noted, shows how domain-specific semantic knowledge can guide predictions in oceanography. These systems echo a broader design pattern: domain-specific LLM solutions often combine generic LLMs with domain-aware classifiers, knowledge bases, and downstream models, but remain targeted at narrow tasks (QA, prediction) instead of general data-science practice.

Zhang et al.'s DeepAnalyze is one of the first explicit attempts to build an agentic LLM specialized for data science itself. They distinguish between domain-specific LLMs that target individual tasks—such as data-science code-generation models, tabular LLMs, and database-oriented models—and workflow-based data agents that orchestrate closed-source LLMs through manually designed procedures (e.g., ReAct or AutoGen). DeepAnalyze-8B is trained via curriculum-based agentic training and data-grounded trajectory synthesis: Zhang et al. construct interactive environments in which the model learns actions such as "Analyze," "Code,"

“Execute,” “Understand,” and “Answer,” and they synthesize multi-step trajectories from simple data preparation to open-ended research questions. DeepAnalyze-8B internalizes these trajectories and is shown to perform strongly on structured data benchmarks and open-ended data-science tasks, making it a promising candidate for a domain-specific data-science LLM. However, it is still a research prototype rather than a widely adopted standard, and even Zhang et al. frame their work as a step toward, rather than the realization of, fully autonomous, domain-specialized data-science models.

Across all these works, a consistent picture emerges. Surveys by Ling et al. and Wang et al. show that in domains like healthcare, finance, and geoscience, domain-specific LLMs are already commonplace and carefully evaluated. Fang et al. identify tabular reasoning as a core technical requirement, and Sun et al. and Wang et al. document an increasingly rich ecosystem of LLM-based data-science agents. Benchmarks such as DS-1000 and DSBC precisely characterize where generic models fall short on realistic data-science tasks. Agents like Data Interpreter and DS-Agent, and frameworks such as CAAFE and MatPlotAgent, demonstrate how far general LLMs can be pushed with scaffolding. DeepAnalyze-8B shows that an agentic, domain-specialized data-science LLM is feasible, but it is early and not yet the default tool for practitioners. As a result, unlike medicine or finance, data science still lacks a widely recognized, domain-specific LLM that reliably supports the full data lifecycle—from understanding messy datasets to designing models, evaluating them, and communicating results. Closing this gap is the central opportunity for work on domain-specific LLMs for data science: to convert today’s patchwork of benchmarks, agents, and task-specific models into an integrated, domain-specialized foundation that genuinely “thinks like” a data scientist.

Chapter 2 Data and Project Management Plan

2.1 Data Management Plan

The system follows a very diligent data management process that is designed to support large-scale dataset construction, preprocessing, model benchmarking, and fine-tuning for domain expert large language models. All the data operations in the system was structured to make sure of integrity, reproducibility, and scalability as it is a central role in high-quality, domain specific data.

Data Sources and Collection

We collected data from various publicly available and open source repos that covers specifically three domains, i.e mathematical reasoning, programming/code generation and theoretical classification and conceptual knowledge. The datasets collected from these sources was transformed to a unified JSON conversation style instruction format which is a good fit with model large language model training pipelines while at the same time maintaining domain-specific structural requirements like LaTeX formatting for mathematics and strict indentation for code.

Data Storage and Organization

We use a dedicated Google Cloud Storage (GCS) bucket to store all the raw and processed data as GCS buckets provides a centralized, scalable, and fault-tolerant storage. The raw datasets are stored in an immutable form to maintain its origin. Intermediate and fully processed datasets are separately stored and they are labeled according to its respective domain,

preprocessing stage, and version. Having such a layered storage structure make sure there is full traceability from raw data ingestion to model-ready training files.

Data Preprocessing and Version Control

Data cleaning, normalization, de-duplication, structural validation and instruction standardization are some of the preprocessing operations done in this project. Each domain that is preprocessed follows a set of strict constraints:

- The mathematical datasets maintains symbolic notation and multi-step solution reasoning.
- The code datasets maintains indentation, hierarchy and syntax integrity.
- The theory datasets were normalized into standard instructional formats.

All the preprocessing scripts were executed in a reproducible notebook pipelines with outputs versioned tracked to make sure there is not data leakage into evaluation benchmarks.

Data Security and Access Control

Even though the datasets that are used for the project are publicly available and contain no personally identifiable information, we still enforced access control policies through Google Cloud Identity and Access Management(IAM). This resulted in a controlled collaboration and minimized unauthorized alterations.

Backup, Recovery, and Reproducibilty

We made sure that the backups are automated and were maintained for all the critical datasets using cloud-based redundancy. Versioned datasets was rollbacked to previous states in the event of preprocessing errors or system failures. The datasets transformations, training inputs

and eval splits were all reproducible using tracked scripts and configuration files stored in the project repository.

Data Sharing and Lifecycle Governance

The last collection of datasets for training, validating and testing were all made available in private secure cloud storage so that we could all work together to experiment and create benchmarks for future research and experimentation. Each of these datasets progressed through a set of controlled phases during their lifecycles, namely: ingesting unprocessed/raw data, pre-processing it, building models (training) and finally, assessing the outcome of the experiment model. This created a thorough end-to-end pipeline of reliable data from start to finish.

2.2 Project Development Methodology

The approach taken in this project is based on a structured and iterative process of building large machine learning applications and uses multiple techniques from MLOps and experimental workflows. As such, it will allow us to build datasets systematically, provide for the repeatable training of models, ensure the evaluations of those models are managed in a controlled environment and will lead to the generation of 'production-ready' deployments for these models. All of the stages in the pipeline have been designed specifically to enable scalability, traceability and continual refinement.

Problem Definition and Domain Scoping

The first phase in the project is identifying the proposed research objectives for the specific domain for our language models. There are three domain-specific models in our project, Mathematics, Coding and Theory Classification that were all created based upon very specific functional and technical requirements identified in the first phase of this project. To create these models, there are mainly four criteria that were established, i.e: (1) Range of Data, (2) Range of Preprocessing Standards, (3) Benchmarking Integrity, and (4) Performance Criteria of fine-tune language models using cloud-based GPU infrastructure. The evaluation and

the criteria of success in this project were defined to provide a baseline for measuring performance all around the project.

Data Collection and Standardization

In the second phase of the project, there is an arrangement of carefully selected open-source datasets with respect to the domain and transformation of these datasets into a single unified JSONL instruction-based conversation style format. On top of that, all the model candidates were aligned architecturally using various standardization techniques while sticking to structural constraints which includes mathematical symbols, LaTeX Notation, and programming syntax. This phase concludes by allowing the establishment of the groundwork necessary for a compatible method of preprocessing and training.

Data Preprocessing and Validation

The preprocessing of the data was done using a controlled, reproducible, step-wise process that consisted of:

- Cleaning the data and removing the duplicates
- Authentication of the structure of both the symbolic and syntactic content
- As per the provided instructions, the data was normalised and formatted
- Each data set was checked for its integrity with respect to the domain of data

All processed datasets were checked for their accuracy before they were separated into training sets, validation sets & test sets.All evaluation benchmarks were kept completely separate from training data to avoid any leakage of information.

Baseline Benchmarking and Model Selection

To compare and experiment with multiple open-source implementations of LLMs, standard benchmarks, such as the GSM8K multi-market series testing and benchmarking conferences (GSM8K & HARD ML), were used. Additionally, zero-shot performance was

measured using quantitative metrics and qualitative LLM grading to identify the best-performing systems per domain. Based on their respective analytics, strong results provided the basis for determining "best" candidates for fine-tuning based on their ability to produce consistent results based on logical reasoning or task completion reliability.

Parameter-Efficient Fine-Tuning

LoRA fine-tuning techniques were used to create a number of models by utilizing Transformer and Unsloth on a Google Cloud Platform GPU instance. The second stage of the fine-tuning was utilizing memory efficient optimization for each of the selected models while also preserving the ability to generalise to other domains. Numerous aspects of each of the training experiments, like loss curves, stability of convergence and hyperparameters were all tracked using weights and biases for a real-time viewing.

Post-Training Evaluation and Analysis

Once the fine-tuning process is done, the re-evaluation of all the models happens in an isolated benchmark datasets that utilize the same evaluation framework that was used during baseline testing. On top of that, a qualitative judgement was conducted using the LLM-as-a-judge method to evaluate the depth of reasoning, the structure of the output, and how aligned the output is to their respective domains. The showing of each of the models before and after they are done being fine-tuned was then compared both quantitatively and visually, which proves measurable improvement after the fine-tuning process was completed.

Deployment and System Integration

Once the solutions was completed, the fine-tuned models were stored in GCS and was integrated into a FastAPI-based inference service which was hosted on Google Cloud Run. To make sure that each of the users were processed by the right domain-specific model, we used an

intelligent routing function to automatically direct the requests to the correct answer based on the prompt. Before delivering the final results for each requested transaction, Llama Guard was used to create an additional safety mechanism to screen for any inappropriate or dangerous input and output data.

Iterative Refinement and Validation

The design of the system included Continuous Loops for Dataset Quality Evaluation, Parameter Fine-Tuning, and Deployment Logic Development to support Continuous Evaluation and Improvement. Each time an Iteration is completed, the Performance of the system is evaluated, and an Error Analysis is completed. Based on these two evaluations, the need for Retraining will be determined. Continuous Evaluation and Improvement through Iterative Refinement will help ensure the Continuous Robustness, Reproducibility and Extensibility of this System over time.

2.3 Project Organization Plan

The purpose of the organization in developing this project was to enable effective collaborative working relationships and to foster the production of modular features. Furthermore, there was a need for each component of the overall project to have clear ownership defined within the project's organizational structure. Given that the project included the data engineering process, model training and adjustment, the deployment of results to the Cloud and assessing model performance, a role-based organizational model supported by cloud-native applications, source control systems (such as Git), and iterative development methodologies provided the best environment for this project's success.

Team Structure and Role Allocation

All of the team's members had well-defined and documented roles and responsibilities based on their individual skills. The main roles assigned were as follows:

- The Data Engineering and Preprocessing Lead had the following duties: to curate data, clean (preprocess) the dataset, normalize the data, validate the structure of the data, and format the data into JSONL format.
- The Model Evaluation and Benchmarking Lead was responsible for developing and executing a zero-shot benchmarking process, selecting candidate models for evaluation, and analyzing the results of their baseline performance.
- The Fine-Tuning and Optimization Lead was responsible for creating a training pipeline for LoRA
- The Cloud Infrastructure and Deployment Lead provided oversight of all Google Cloud resources, training endpoints, and Cloud Run deployments associated with running AI workloads on Google Cloud (GCP).
- The System Integration and Safety Lead handled the integration of the backend FastAPI application with the NPR routing logic; implemented the Llama Guard for safety enforcement.

This way the team was held accountable and permitted to work on their own technical components concurrently, allowing for efficient development across all technical components throughout the project.

Team Structure and Responsibilities

To prevent model training bottlenecks, the team has been structured with a modular role-based approach allowing each team member to have a defined responsibility for both a specific

domain and its fine-tuned model. By doing so, the workload of the team members is evenly distributed, with no team member carrying an excessive workload.

Each team member is responsible for their specific function, but has the following responsibilities in achieving an overall team goal:

- Manjot : The individual responsible for ingesting, cleaning, deduping, and maintaining consistency of all processed datasets in the Theory, Code, and Math pipelines, and creating a common formatting standard (JSONL).
- Darpan & Saqib : The individual responsible for creating baseline evaluation datasets, creating and analyzing the distribution of the dataset message lengths, and providing post-training evaluation reports. The evaluation datasets were maintained separately from the training datasets in order to ensure fairness.
- Sai: Responsible for configuring Google Cloud Storage (GCS), configuring endpoint deployment and integrating Cloud Run, to maintain the API endpoints as a stable service by using FastAPI.
- Prathyusha: Responsible for connecting all the components of the pipelines and implementing the safety layers to maintain the consistency in formatting across the domains.

All team members trained one specific model each rather than having one team member solely responsible for training all of the models, including:

- Running LoRA fine-tuning on GCP
- Configuring the training parameters and hyperparameters, and maintaining records/logs
- Monitoring and adjusting the Loss Curves during the training
- Exporting the LoRA Adapter for later use

- Providing the results of the training for evaluation and comparison to each other

Workflow Coordination and Collaboration

The partnership utilized an established repeatable process in a cyclical format:

- Weekly meetings-Planning weekly to keep working on datasets, check the status of each other's training progress, and prepare for deployment together
- Share Notebooks On GCS / Collab To Be Able to Create Reproducible Pipelines.
- GitHub Versioning, Branches, Pull Requests, and Reviews.
- Accountability- Each team member is responsible for verifying the accuracy of each other's data log and training logs.

By the end of this structure, while all the team members were training their own models independently, the whole project was kept in sync throughout, so that everything was integrated with each other and worked together.

Task Management and Milestones

1. Ingest and validate the raw datasets
2. Preprocess, deduplicate and format the raw datasets
3. Evaluate the models using a zero-shot baseline
4. Fine-tune the individual LoRA models (one for each person)
5. Evaluate and compare the models after fine-tuning
6. Deploy the models into GCP
7. Validate and report on the final system

For every milestone, there were verification checkpoints to ensure that the quality of the models was acceptable, and to determine if the models were ready to be deployed or if additional steps needed to be taken before proceeding.

Tools and Collaboration Platforms

- GitHub, the primary source control and issue tracking system.
- Google Cloud which provided the team with storage, compute, and deployment capabilities.
- Colab for preprocessing and developing model architecture.
- Discord for communicating among team members.
- FastAPI for inference back-end development.
- LlamaGuard for filtering out potentially unsafe inputs to the model.

Risk Management and Contingency Planning

- The preprocessing output was versioned and able to be reverted if any failures occurred
- Training check points were created for the entire duration of the training
- Deployment Endpoints were tested prior to integration into production
- Each member had another reviewer for their model workflow layout
- In the event of a GPU quota issue with training jobs, alternate geographical regions were used for retraining.
- In this way, there was never any single model training that could delay the overall progress of the model.

2.4 Project Resource Requirements and Plan

This project is based on a coordinated collection of software, Process Resources, Unit Compute Resources, and Cloud Storage, that supported every step in the workflow - from acquiring the dataset, processing it, exploratory analysis, LoRA Model training and evaluation, and Back End Integration. The resource plan has been created to be a Cost-Effective, Repeatable

and Accessible resource plan for a decentralized student team to support Realistic Machine Learning workloads.

Computational Resources

Colab was the primary environment for updating and testing LoRA, and the following tiers were used by the project.

- T4 GPU instances (provided baseline for fine-tuning)
- L4 or A100 GPUs (if available through Colab Pro), with
- 16-24 GB VRAM and
- High (25-50 GB) RAM runtimes.

Through the use of these GPUs, all seven to eight billion parameter models were efficiently fine-tuned and allowed all members of the team to build their own specialty models within a reasonable time and space limit using Unsloth LoRA.

Local Development Machines

Each team member used his or her own computer for the following tasks:

- Developing scripts and notebook files that were used to prepare data
- Performing data analysis and graphical displays
- Running FastAPI backend for inference
- Performing local tests on loading LoRA adapters.

Each team member's minimum specifications included:

- 8-16 GB of RAM;
- Multi-core processors;
- Stable internet connection.

All team members' computers had the minimum specifications to perform all non-training tasks.

Cloud Resources

The shared repository of Cloud Storage was the basis of all of the following types of Storage.

Cloud Storage was used for:

- Raw Datasets (Mathematics, Code, Theory)
- Processed JSONL Files
- Metadata Logs, Statistics, and Visualizations
- LoRA Fine Tune Model Weights.

Reproducible Access to All Team Members Provided by GCS Prevents Version Drift Across Stages of Preprocessing.

Why Choose GCS?

- Easy Collaboration with Google Colab
- Automatic Versioning of Files
- Low Cost to Store
- Centralized Access to Datasets

Software Tools and Libraries

Machine Learning Frameworks

- PyTorch
- Hugging Face Transformers
- Unsloth (for optimized LoRA training)
- Accelerate
- Tokenizers

These libraries enabled efficient fine-tuning of large language models while managing VRAM constraints on Colab GPUs.

Data Engineering Stack

- Pandas and NumPy - They were used for Preprocessing the data into usable, processed datasets
- Parquet I/O - They provide an open-source format for storing processed datasets on filesystems, cloud platforms, and data warehouses.
- Matplotlib / Seaborn - These are libraries used for visualizing processed datasets.
- Scikit-learn - This was the primary tool utilized for Statistical Analysis and Analytics on processed datasets.

The above libraries are commonly used in data Cleaning, Deduplication, Quality checking, and Analysis of Message-Length Distribution.

Tools used for Backend & Deployment include:

- FastAPI - Server for serving model inference requests
- Uvicorn - ASGI (Asynchronous Server Gateway Interface) server for serving FastAPI applications
- Python (3.10+) - Python 3.10 and up are the main versions utilized for building applications with FastAPI, so all libraries must be built using Python (3.10+) to properly execute FastAPI models and libraries.
- LlamaGuard — Filtering mechanism for creating SafeGUARD*
- Ngrok (Optional for testing temporary endpoint)

2.5 Project Schedule

The entire project is anticipated to take place over 12 months from January through December 2025. Each phase of the workflow (i.e., Data Acquisition, Data Preprocessing, Exploratory Analysis, Fine Tuning the Model, Evaluate Model Performance, and Final System Integration) has been scheduled to allow for steady progress towards project completion, as well as to create a reasonable amount of work for each individual within the team.

Below is a Gantt Chart that gives a visual representation of each of the aforementioned phases and their expected durations. Additionally, the Gantt Chart shows how each task will differ from others and how there will be some overlap between multiple tasks during the course of the project. The chart also shows how the phases must proceed in order to complete the project and deliver the final products (Final Validation and Report).

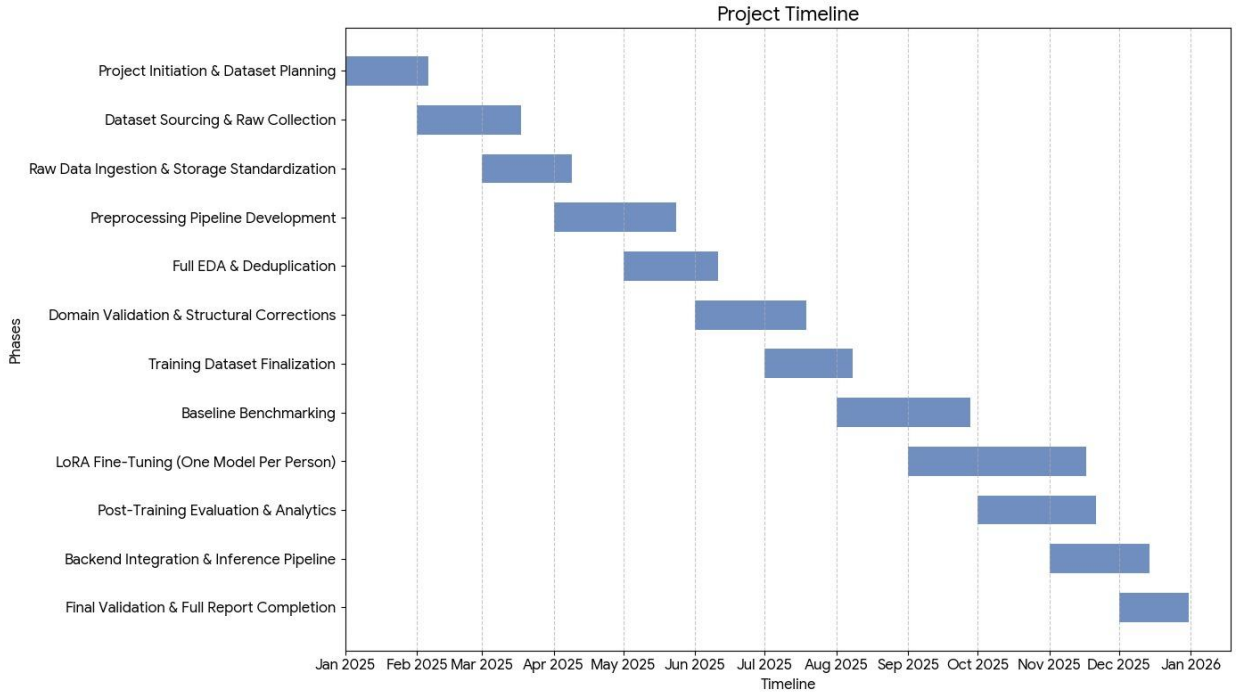

Figure 1: End-to-End Project Timeline (Jan 2025 – Dec 2025)

Original From Team 2

Resource Usage Plan

The Data Acquisition and Preprocessing phase takes place in two parts: using Local machines to inspect original datasets and using the Colab CPU runtime for bulk data processing of files, in addition to using GCS to store interim processed data.

The EDA and Visualization phase takes place in two parts as well, using Local Python scripts to perform smaller visualizations On the other hand the Colab notebooks will perform visualizations of larger histograms and distributions of datasets.

The LoRA Fine-Tuning phase happens through the use of GPU's (via Colab). Each member of each team will work independently on this phase, therefore requiring each member to use their own GPU sessions to be completed. GCS would be utilized to check point the work from the individual model.

Each team completed the Evaluation and Comparison phase on both Local CPU and GPU machines to accurately assess the speed of processing and perform visual analysis of the results on their Local computer.

The Backend Integration phase occurs on Local CPU machines using FastAPI to complete inference to return results and load LoRA models into their base models. The models would then continue through the Safety Validation and routing processes.

Final Documents and Deliverable phase occurs through using Local editing tools and GCS as a shared archive for finished products.

Risk Management

To help minimize resource-related risk:

- If we experience a shortage of GPUs: We use Colab free tier & Colab Professional; use alternate regions
- Data Integrity: All preprocessed output and LoRA weight files have been stored in Google Cloud Storage, complete with version history
- Interruptions During Runtime: All training scripts support resuming and checkpointing
- Local Machines: Heavy workloads are run on Colab while lightweight workloads are run locally
- Team Coordination: By using shared notebooks and maintaining a consistent file structure, we ensure team members work in unison with one another.

Chapter 3 Data Engineering

3.1 Data Process

This project has developed a structured and full pipeline Data Processing workflow, in support of the development of Large Language Models for domain experts in three different domains: Mathematics, Programming and Theoretical Knowledge. The overall goal of the Data Process is to ensure high-quality data, structural integrity, scalability and complete reproducibility of the end product of model-ready datasets across the entire range of processing stages, beginning with the raw acquisition of data through to the final staging of the data into model-ready datasets.

The first step in the data process includes the acquisition of raw open source datasets from multiple reputable sources, and that the original format of those datasets will be ingested

into PaaS (Platform-as-a-Service) based cloud storage to maintain the data provenance, and ensure the ability to trace back to the original source of the data. During this initial ingestion step, raw datasets are treated as immutable. During the pipeline, they will never be modified directly.

After ingestion, a controlled preprocessing pipeline will be applied to clean (eliminate bad entries) and normalize/standardise the datasets into a single format. During this stage, we will enforce domain-specific constraints to maintain the essential structural properties of the datasets. Data sets for mathematics will contain LaTeX expressions, multi-step reasoning, and chained steps; code datasets will contain the proper indentation and syntax; and theoretical datasets will have free-form text normalized into a single instructional format.

Once all preprocessing has been performed, the datasets will be divided into training, validation, and testing subsets. Datasets will be split according to appropriate domain-specific splits to ensure there are no instances of data leakage during training. All datasets used to create benchmarks for this evaluation have had their datasets removed from the training pipeline to ensure there is no interference with the experimental process. All datasets created will undergo logging and archiving for future reproducibility.

The last part of the data process will involve the validation and verification of the prepared datasets. An automated validation check will be performed along with a manual sample verification to check for schema consistency, formatting, structural consistency within domain, etc. Only validated data sets can proceed into the model training stage. This structured data process will provide a consistent high-quality data basis for all downstream model benchmarking, fine tuning, and deployment.

3.2 Data Collection

This phase of data collection was centred on obtaining the best quality and open-source datasets that can be used for training and evaluating the abilities of experts or large language models in the areas of mathematics, programming, and classification. The goal was to obtain datasets that were varied, contained a high degree of data structure and fit into the framework of instructions, while ensuring their legal usability, traceability and for the intended domains.

All Datasets were obtained from publicly available sources (such as repositories, research benchmarks, Open Education) only. The selections were based on the credibility of the datasets, structural completeness, domain alignment, formatting consistency, and all had to be freely licensed for research purposes. Datasets for benchmarking purposes (i.e. GSM8K, MATH, DS-1000) were identified as well, but the decision was made not to use them for training but only for evaluation.

The datasets were ingested in their original format (JSON, CSV, Text) and stored on Google Cloud Storage (GCS) as immutable raw copies so that the provenance of the datasets could be maintained. Each dataset was assigned the following metadata so that they could be tracked and reproduced. Then all of the collected datasets were grouped according to their domains:

- Mathematics: These datasets were used to model symbolic problem solving and multi-step reasoning problems that used LaTeX formulae.
- Code: These datasets were used for instruction-based programming and preserved their syntactical order, indentation, and so forth.
- Theory: Textual datasets are composed of conceptual, definitional, and classification information.

All the datasets were put into the queue once they were ingested, but the raw data layer did not see any adjustments prior to extracting them. Maintaining this separation between the raw data layer and the pre-processed phase was necessary to support the eventual future audit trail of data items and thus retain long-term data integrity.

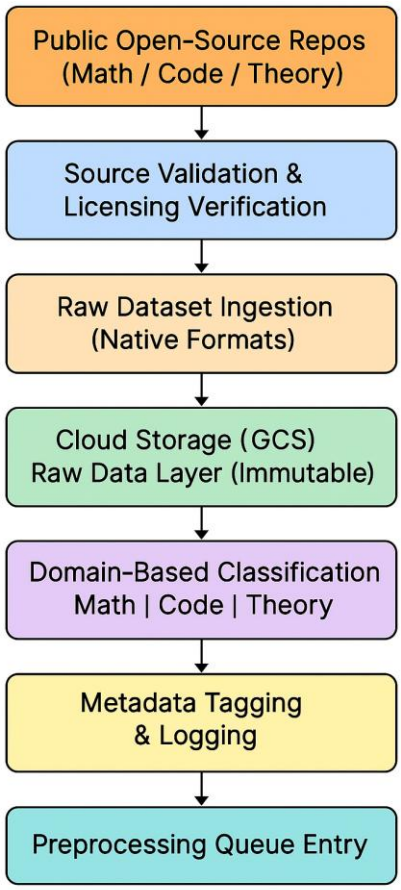

Figure 2: Data Acquisition and Ingestion Pipeline

Original from Team 2

3.3 Data Pre-Processing

The pre-processing stage of the data set is very important because it ensures that every data set collected meets certain requirements based on the type of collection (large-scale training for domain-expert language models) to produce a model that will meet the educational/training

requirements. Due to the unique nature of Math symbols, programming syntax and structured theoretical instructional formats, the pre-processing phase of this project was developed with strict domain-specific requirements and reproducible transformation logic.

All pre-processing procedures were conducted on copies of the raw data stored in the cloud; the original raw data cannot be changed. Data cleaning and data normalization were completed as the first step in pre-processing, which consisted of removing records that had been corrupted or without a valid file structure (the submission of a duplicate), records without a consistent text format and malformed samples. Special attention was given to standardising the encoding types of text to allow for compatibility across different types of training environments..

Once the dataset was cleaned, Preservation of Domain Specific Structures was assured through the application of domain specific structural rules. For mathematics data, It was required that all LaTeX, symbols and multi-step calculation never lose tokens, formatting distortion or unrecognised codes; data contained in Mathematics must maintain the logical structure necessary to maintain an accurate logical outcome. For programming, it was required that all spaces, indentations, logical hierarchies and logical block structures were maintained as they would have prevented the corruption of logic and would also prevented the inability to execute the code; for data theory, content in free form was required to be reduced to a standardised instruction-response format while retaining Clarity of Concept(s).

Subsequently, the formatted datasets were converted into a unified conversational instruction format represented as a prompt-completion pairing, within a structured JSONL element, to support instruction tuning; this structured format will offer uniform architecture across transformer-based training models and parameter-efficient fine-tuning approaches such as LoRA. After formatting, validation through schema verification checks, as well as manual stratified

sampling, were performed to identify consistency of formatting, domain accuracy, and structural integrity; records that failed validation were corrected or removed from the datasets. Once the datasets were completely processed, they were versioned, stored with metadata, and moved to the data preparation stage, which further refined them for use in training, validation, and testing. Thus, the controlled and reproducible format-processing pipeline provides the foundation on which to create high-quality and structurally sound data for all subsequent benchmarking, fine-tuning, and deployment efforts.

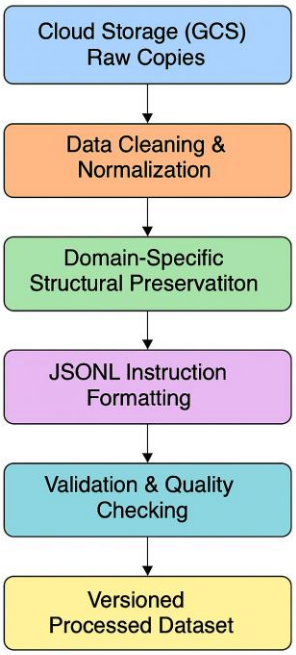

Figure 3: Data Pre-Processing and Transformation Pipeline

Original From Team 2

3.4 Data Transformation

The transformation phase converts completely pre-processed data sets into structured, optimized formats for use in Large Scale Model Training, Evaluation, and Deployment. This phase is concerned with the transformation of validated data into Machine Learning Ready Formats by preserving all of the domain specific semantics and syntaxes.

After successful pre-processing and validation all datasets were transformed into a uniform instruction response schema compatible with transformer architectures. Each data record was mapped to a standardized input output framework supporting instruction tuning. Therefore maintaining consistency in learning across different families of models. This transformation allowed for architectural compatibility across all two selected base models used in this project. The used mathematical datasets were transformed while retaining symbolic expressions, LaTeX Formatting, and Multi-Step Reasoning Chains aligned to the standardized instructional template. Code datasets were transformed by embedding code blocks into structured outputs without changing the indentation level, execution logic, and syntactic hierarchy. Theory datasets were transformed from unstructured conceptual descriptions into formal instruction prompts and Provide Clear Expected Responses.

Other Transformations Completed Included:

• Removal Of Residual Placeholders And Formatting Artifacts

• Standardization Of Special Tokens And Control Sequences

• Whitespace And Line Break Normalization To Ensure Training Stability

After they had been transformed, the datasets were indexed and assigned domain-specific identifiers, allowing for linking raw sample to pre-processed record to transformed training entry, retaining traceability through the process.

The transformed datasets were stored as version controlled JSONL files in the cloud storage environment and progressed to the data preparation step in the final assembly and split of the overall dataset. This transformational processes will allow for all model input to have structural

integrity, semantic accuracy and to have optimized the model for parameter efficient fine-tuning and large scale inference.

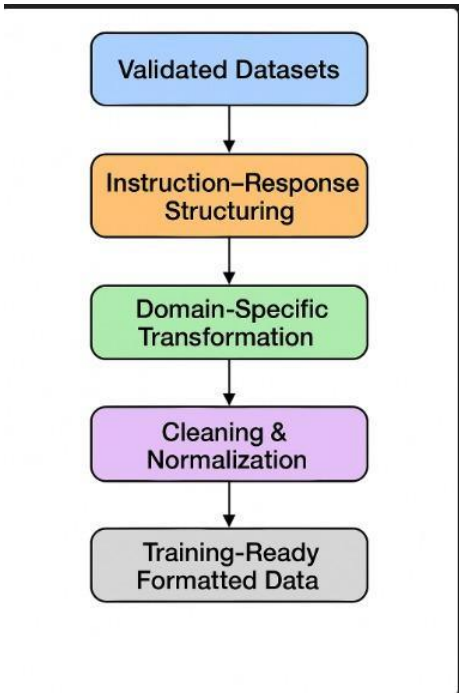

Figure 4: Dataset Structuring and Training Preparation Pipeline

Original From Team 2

3.5 Data Preparation

All datasets were compiled, organized, and validated in the final preparation step prior to building the model, culminating in these organized datasets being placed into structured training pipelines. That is, the preparation phase will prepare all datasets to ensure they have been prepared appropriately. Additionally, these datasets will be partitioned, balanced, and packaged appropriately for efficient, large-scale, fine-tuning of models on GPUs.

As they were transformed to the unified instruction–response format, all datasets were organized categorically by domain. Each domain category underwent evaluation to ensure they are structurally consistent (e.g., have the same number of columns), contain valid tokens, and are

free of any formatting artifacts from the previous stage. Each dataset for each domain was then further divided into three partitions: training, validation, and testing partitions. Each partition strategy ensured that no data leaks between training and validation/testing partitions and preserved the domain's diversity across partitions. Benchmark datasets (e.g., GSM8K, MATH, DS-1000) were kept exclusively for evaluation; thus, these datasets were not used for the training phase.

At this point in the production of the training data, additional procedures were implemented to perform dataset balancing and to limit the effect of domain shift, oversampling or undersampling, and instructor bias. The mathematical datasets were balanced by identifying difficulty levels and symbolic density; the coding datasets were balanced by identifying the types of problems and languages used; and the theory datasets were balanced by identifying the type of concept.

Next, each dataset subset was labelled, indexed, and stored as versioned JSONL files in the cloud. Metadata was also recorded for each dataset subset, including the number of samples, category of domain, token statistics, and version of the dataset transformation. The purpose of recording this metadata was to provide assurance that downstream training experiments could be traced back to the exact dataset used for fine-tuning.

Finally, the prepared datasets were loaded into a training format compatible with GCP, where the datasets were prepared into the format of dataloaders created to accommodate batch processing, tokenization, and parameter-efficient training using LoRA. By the end of this phase, the data pipeline had produced structured, validated and ready for training datasets that form the basis for consistent and high-quality model fine-tuning.

3.6 Data Statistics

The processed datasets were thoroughly evaluated with a robust statistical approach in order to measure the quality of the dataset, distribution within domains, structure, and whether or not the datasets met the requirements for LoRA-based fine-tuning. In addition to being aggregated, the domains of each dataset were preprocessed in a way specific to their intended purpose. The sorted data sets were deduplicated to eliminate duplicate entries, which contributed to more efficient training. Thus, the final dataset sizes include only validated, unique, and instruction-ready examples.

Final Dataset Sizes after Deduplication

The following domain totals came from the preprocessing pipeline:

• **Theory Domain - 1,657 examples**

The Theory dataset includes a set of conceptual, definitional, and higher-order reasoning prompts. After deduplication, we eliminated 42 duplicate entries (from 1,699 to 1,657).
Though it is less extensive than the other datasets, the density of concept representation makes it well-suited to fine-tune both Falcon-H1 and Qwen-Theory models.

• **Code Domain - 19,443 examples**

The Code Dataset contains structured coding instructions and Code Generator tasks.
After deduplication, 92 duplicates were eliminated (from 19,535 to 19,443).
Due to its increased number of examples and syntactic heterogeneity, this dataset was a strong base for fine-tuning SeedCoder - 8B.

• **Math Domain - 138,115 examples**

Math Dataset is by far the largest dataset, it contains symbolic notations and multi-step reasoning problems including LaTeX-format solutions.
Through deduplication, we eliminated 7,924 duplicated entries (from 146,039 to 138,115).

The size and complexity of this dataset facilitated successfully fine-tuning both AceMath - 7B and Qwen-Math - 7B for applications that require sophisticated mathematical reasoning.

Total Across All Domains: 159,215 Processed Examples

This reflects the final training corpus used in all LoRA fine-tuning experiments.

Token Length and Structural Patterns

The finalized training dataset used for training LoRA in all experiments is reflected here:

Math Domain:

- Longest average sequence length,
- Highest symbolic density (i.e. contains the most LaTeX markup, operators, superscripts)
- Most structurally complex.

Code Domain:

- Average length of sequence of tokens was of medium length/size.
- Tokens generally had predictable syntactic patterns.
- The way in which tokens were issued also depended heavily on the use of proper indentation and white space.
- The distribution of tokens issued across programming structures was uniform.

Theory Domain:

- Short sequences of tokens that contain natural language instruction.
- A wider variety of vocabulary.
- Greater variation in the way instructions were expressed in unique phrases.
- These distinctions aided in establishing LoRA-specific guidelines for Max Seq Length and Max Batch Size for all three domains.
- **Quality/Validation & Filtering**

For validation and filtering, all of the data from each domain was validated against structural and semantic checks which correlated with the domain being validated (e.g., theory for instruction-response alignment and low-quality conceptual duplicates removed; code checked for syntax integrity by way of validating spaces and indentation; and math compared to ensure LaTeX consistency and validated. All samples that passed through the validation steps above were included as part of the final processed dataset.

Domain Proportions

After data had been deduplicated, the Three-Database final dataset exhibited the following domain distribution:

- Math: 86.8%
- Code: 12.2%
- Theory: 1.0%

This distribution naturally reflects the high demand for math-specific data and the availability of large open-source datasets specifically for mathematics.

Summary of Readiness to Train

The final dataset contained the following:

- Cleaned, deduplicated, and standardized samples/examples.
- All samples/examples in the final dataset were formatted to comply with the JSONL format (instructions).
- The structural integrity of all domain-specific structures exhibited a high degree of fidelity to the unique characteristics of their respective domains.
- The token distributions contained in the final dataset were suitable for use with LoRA fine-tuning.

3.7 Data Analytics Results

After processing and removing duplicates from the datasets, exploratory data analysis (EDA) was performed in order to evaluate the characteristics of message length, complexity of domains, and overall distribution of the dataset.

The interpretations that are presented below all correspond to the same graphical visualizations.

Message Length Distribution Across Domains:

Theory Dataset

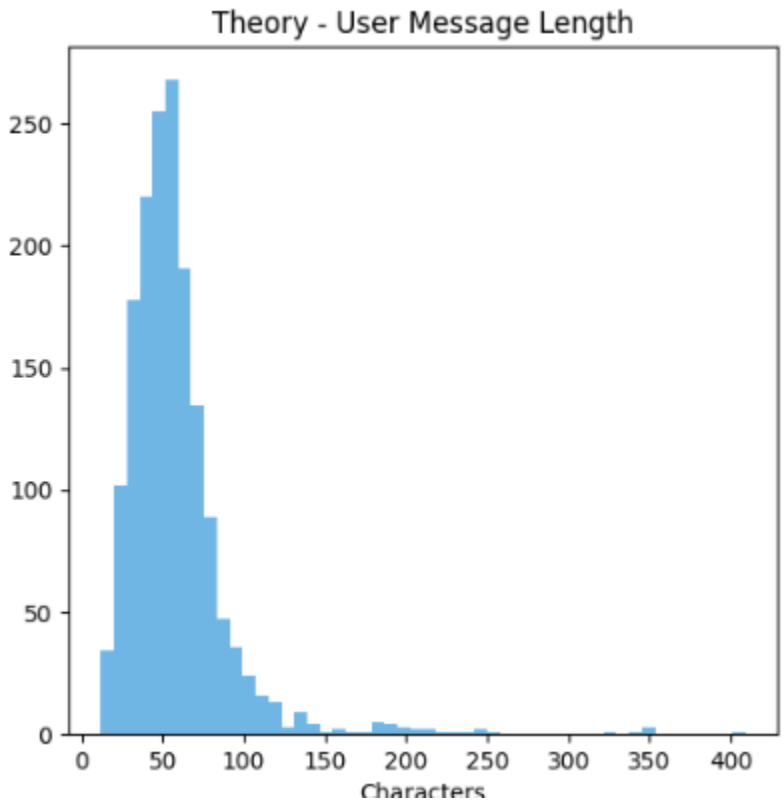

Figure 5: Theory - User Message Length

Original From Team 2

The histogram above demonstrates that the average user-generated messages within the Theory domain tend to be very short (typically) ranging from approximately 20 to 80 characters long with very few exceeding 150 total characters.

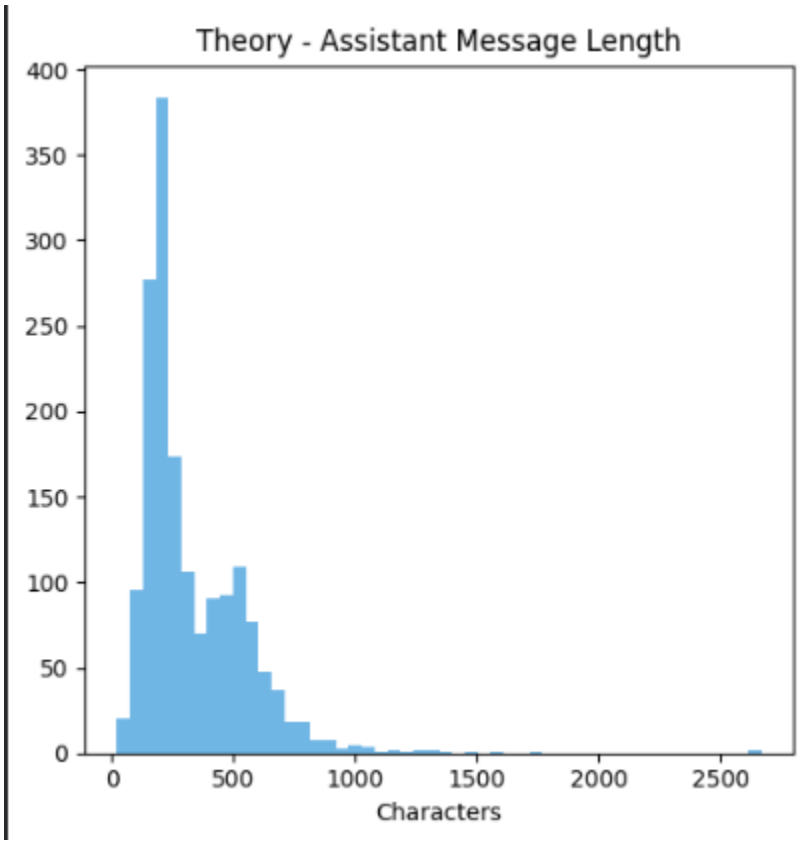

Figure 6: Theory Assistant Message Length

Original From Team 2

User-generated responses tend to be longer in length than that of the original user input; however, user-generated responses are typically limited to 100 to 600 characters long.
This mirrors the nature of conceptual terminology in the domain and the requirement of minimal explanation and reasoning for providing concepts.

Code Dataset

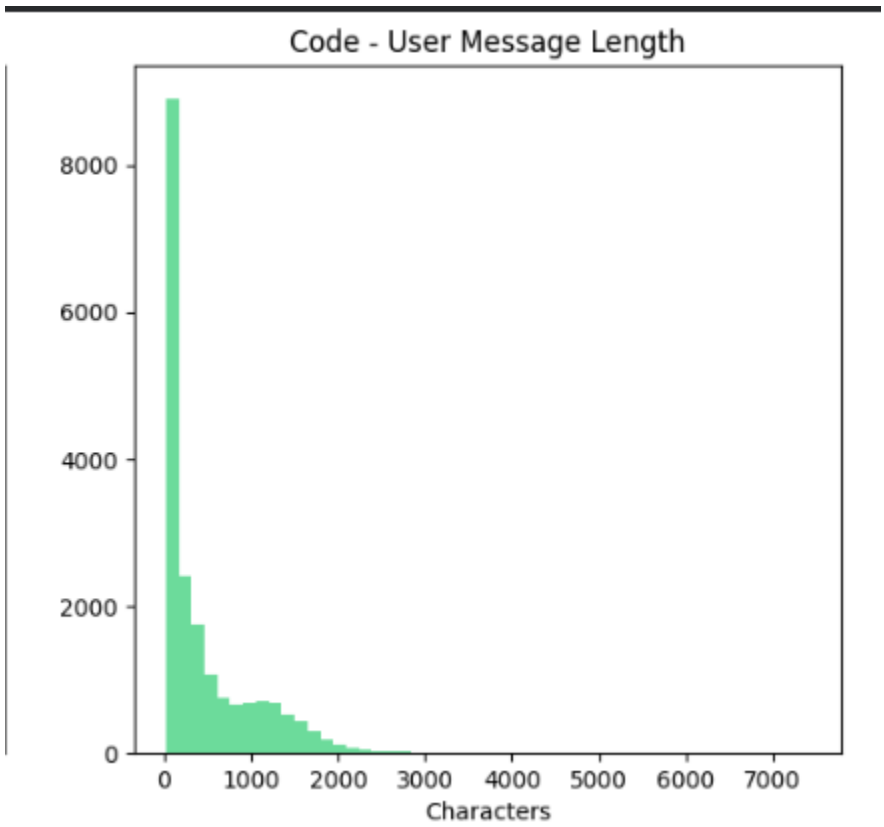

Figure 7: Code - User Message Length

Original From Team 2

With input from users, a lot of codes are longer and have descriptions of the problem or examples embedded within them, but some codes are over 500 characters in length and the tail on those codes can be up to 2000 characters long or more.

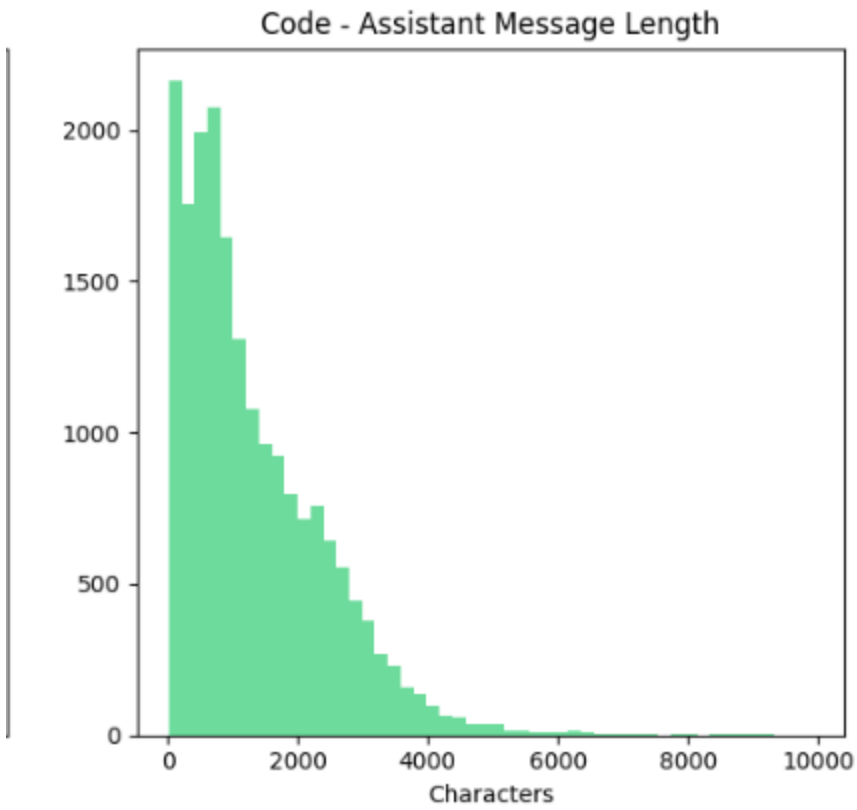

Figure 8: Code - Assistant Message Length

Original From Team 2

The output from the assistant is typically 1,000+ characters for one line of code and/or 3,000+ characters (due to the number of lines of code), including code snippets, formatted outputs, and explanatory text.This long-tail of output length from the assistant corresponds to the structure of programming tasks' complexity

Math Datasets

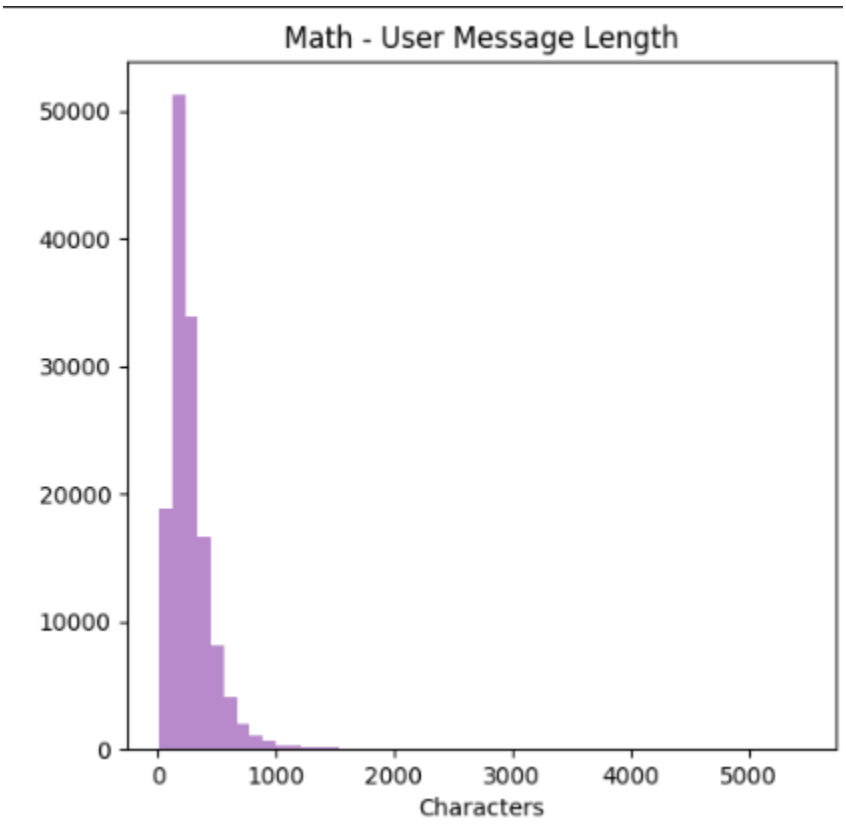

Figure 9: Math: User Message Length

Original From Team 2

User Messages related to Mathematics have the widest variance in length, typically being 200-1000+ characters in length due to the use of Mathematical Symbols, LaTeX Formatted text, and Multiple Steps in a question.

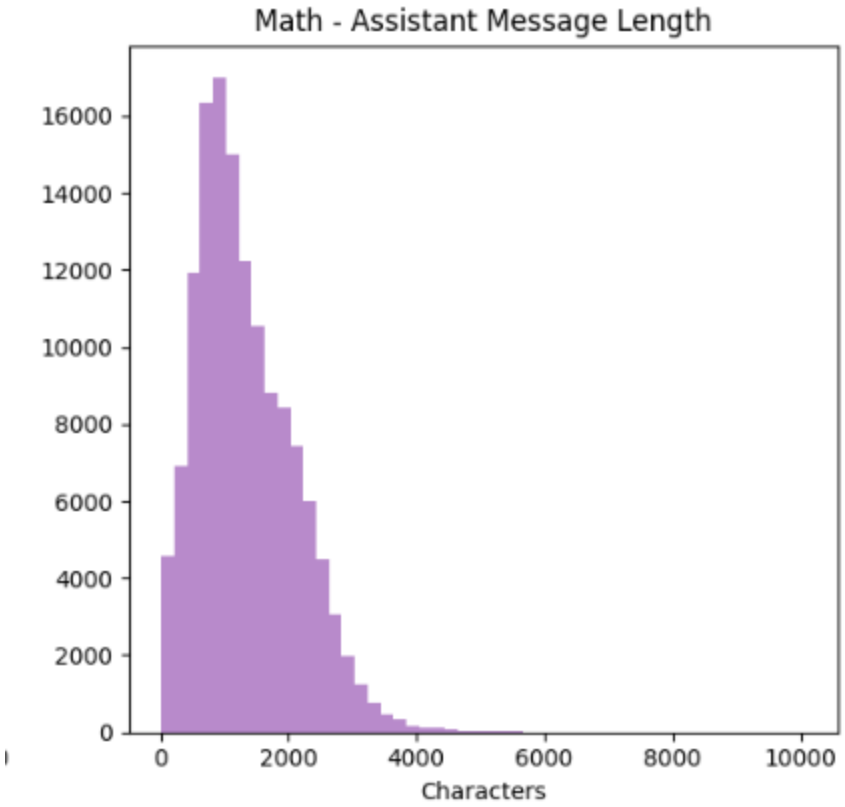

Figure 10: Math - Assistant Message Length

Original From Team 2

Responses provided by Assistants are typically in a character range of 1000-2500 characters, with step-by-step solution examples, derivation examples, and the use of complex LaTeX markup, resulting in a cluster around 1000-2500 characters.

Post-Processing Dataset Distribution

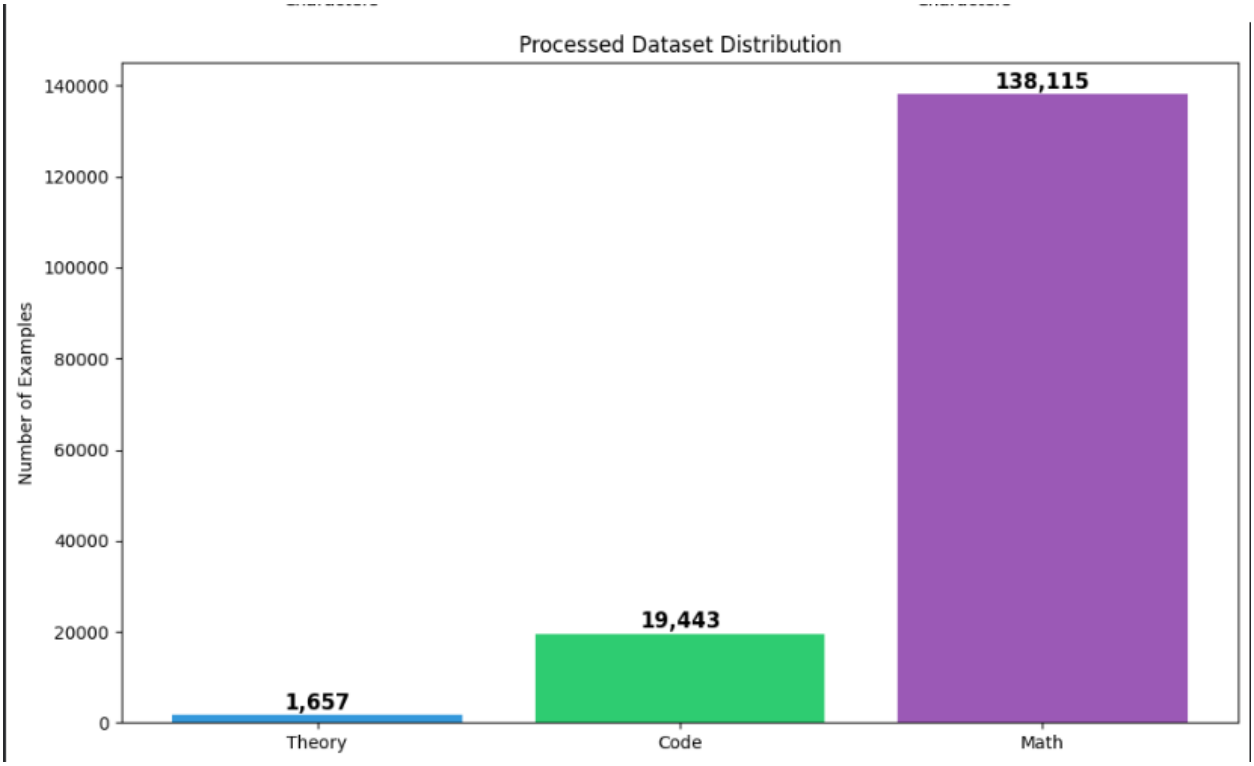

Figure 11: Processed Dataset Distribution

Original From Team 2

- The final counts of deduplicated datasets are represented in the bar chart above. In the mathematical dataset, the numbers are as follows: 138,115
- The overall count of codes (codebooks and product codes combined) is: 19,443
- For the theoretical dataset, the count equals: 1,657

Mathematics, as shown by these numbers, has by far the largest amount of data supporting this conclusion. As expected, this is an indication of the importance of mathematics in performing symbolic reasoning operations.

Insights for Model Training

1. To set the maximum length for sequences due to the long tail distributions noted in the math and code datasets:

- Tuning of higher max_seq_length
- Adjusting batch sizes so they fit within GPU memory limits
- Fine tuning the LoRA rank and alpha settings for longer context learning, will help find better representations of the longer sequence.

2. Since the clarity of the distributions gave us consistent ways of viewing the domain (theoretical) it was possible to have stable fine tunings which means reliable fine tuning of concepts. The predictable structure of the code allows us to always be able to retrieve the same format.

3. The fact that there were no extreme outliers on the histograms is an indication of strong clustering and therefore confirms that the data has been:

- Preprocessed correctly.
- Not subjected to catastrophic format damage.
- All domains are very well structured.

Overall Interpretation

Analytics show the dataset offers the following properties:

- Diversity of structure across domains (urban areas, forums, etc.)

- Large, very large and articulate amount of math.
- Syntactically dense when coding for all.
- Offering a narrowly defined and representative set of theoretical concepts.
- Fully Supporting the development of fine-tuning models through LoRA.

All of these factors combine to create models trained using a model that is a true representation of how humans actually write instructions, do computations and how domains reason about issues.

4 Model Development

4.1 Model Proposals

Creating a smart multi-domain system able to do math reasoning, write code, and give coherent theoretical explanations necessitated choosing and pointing out large language models (LLMs) with specialization. Every one of the three target areas mathematics, programming, and theory requires different reasoning techniques. Hence, one universal model could not be relied upon to produce good performance across all tasks. On the contrary, the system designed itself with a domain-specific approach by using a number of pre-trained models, each recognized for its competency in one area.

The models in mathematics were selected according to their widely acknowledged capabilities to perform various tasks like symbolic manipulation, structured derivation, multi-step numeric reasoning, and so forth. The resulting selection was composed of Qwen2.5-Math-7B-Instruct, AceMath-7B-Instruct, WizardMath-7B-V1.1, DeepSeek-Math-7B-Instruct, and Llemma-7B. These choices were based on the models' previous performance on the abovementioned benchmarks of GSM8K and MATH and their exhibited proficiency in deep chain-of-thought

reasoning. The system utilized models that were specialized for algorithmic reasoning, code synthesis, and debugging for its programming tasks. Among such models were Seed-Coder-8B-Instruct, Qwen2.5-Coder-7B-Instruct, DeepSeek-Coder-6.7B-Instruct, CodeLlama-7B-Instruct, and CodeGemma-7B-IT, etc. The models were first trained on huge code datasets and evaluated by the HumanEval and DS-1000 datasets.

In considering theoretical reasoning, the system directed its attention towards the models which could perform abstraction, give concepts, and structured summaries as well. The candidates for this included Falcon-H1-7B-Instruct, Qwen2.5-7B-Instruct, Gemma-2-9B-Instruct, Llama-3.1-8B-Instruct, and last but not least, Mistral-7B-Instruct-v0.3. Collectively, this meticulously selected group of models was the groundwork for a modular framework where each model could be enhanced and utilized for the specific area it was best at.

4.2 Model Supports

Training Frameworks

The models that were chosen were combined and adjusted through the use of a standardized infrastructure that was meant for modular experimentation as well as efficient adaptation. The library of Hugging Face Transformers was applied for all training and inference processes together with LoRA-based fine-tuning through the PEFT framework. LoRA granted updates to only a small fraction less than 2% of a model's parameters, thus retaining the core pre-trained knowledge while at the same time making the models suitable for specific domains. The fine-tuning was done in a compute-efficient environment, hence there was no need for distributed training or external memory optimization techniques.

The first training phase was executed on Google Colab Pro+ equipped with A100 GPUs having 40GB VRAM, this made it possible to run the entire 7B-9B model with LoRA for 7B-9B models' full training. Evaluation experiments and smaller ablations were carried out on Kaggle's T4 and P100 GPUs. The preprocessing tasks of cleaning, tokenization, and formatting of training samples were done on local CUDA-enabled systems. Google Cloud Storage was used as a central point for all artifacts including checkpoints, training logs, and tokenizer configurations to ensure reproducibility, easy access from multiple devices, and safe continuation of long-running sessions.

The preprocessing pipeline included domain-specific constraints: for math datasets, symbolic notation such as LaTeX was preserved; for code datasets, formatting, indentation, and execution context were enforced; and for theory datasets, noisy text artifacts and formatting inconsistencies were cleaned. A baseline zero-shot evaluation was performed for all candidate models in each domain prior to fine-tuning. Based on this reference, the most promising models were selected for LoRA-based transfer learning. A validation set was used throughout training to monitor accuracy, reasoning consistency, and overfitting. This high-level architecture ensured that every model in the system had a dedicated pipeline for development, training, evaluation, and deployment.

Artifact & Experiment Management:

All model checkpoints, configuration files, training logs, and outputs produced are kept in Google Drive or Google Cloud Storage. This enables experiments be reproducible, accessible from different devices, and long-running ones to be safely resumed.

High-Level System Architecture

The complete system architecture is a discipline-sensitive pipeline from the raw data to the final models of the ensemble in a well-structured manner. The procedure commenced by raw information coming from the Math, Code, and Theory. These are initially subjected to a preprocessing stage that domain-specific constraints: LaTeX and math symbols are kept for the math datasets; the indentation, code blocks, and newline formatting are strictly followed for the code datasets; and the theory datasets have noisy artifacts like placeholders or inconsistent text patterns cleaned up. After the preprocessing, the data is divided into domain-specific training, validation, and test splits. We perform baseline zero-shot evaluations across all candidate models within each domain to establish reference performance without any fine-tuning; these are established as the maximum performance possible without any adjustments. On the basis of these metrics, we then select the best models, one from each domain, for transfer learning. The selection of models is further accompanied by a LoRA-based pipeline for fine-tuning, followed by a comprehensive evaluation stage where the models performance is measured against the entire pool of the best performing models. The models that have exhibited the best performance are either integrated into the final ensemble or selection mechanism, thereby guaranteeing the strong coverage of Theory, Math, and Code tasks.

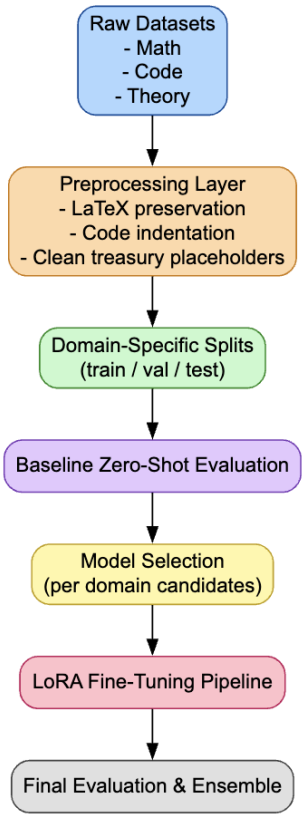

Figure 13: End-to-End Specialist Model Development Pipeline

Original from Team 2

Data Flow Diagram

This system logical data flow can be easily used as a chain of processing stages: data acquisition, cleaning, tokenization, model input construction, training, evaluation, and deployment. To be more precise, the controlling stage of the raw data processing is to clean the data so as to eliminate the invisible part of it (noise) and to apply the domain-specific formatting constraints. The subsequent stage involves the transformation of the cleaned text into subword units, which are model-compliant and are organized into input-output pairs that are appropriate for each task. The training loop then receives these tokenized samples, where the models are fine-tuned via LoRA on domain-specific splits while validation metrics are monitored. After the

training is completed, the models are evaluated through both automatic benchmarks and LLM-based judging to assess correctness and reasoning quality. Ultimately, the highest-performing models are either prepped for use or bundled together into an ensemble that channels inquiries to the model specialized in the corresponding domain.

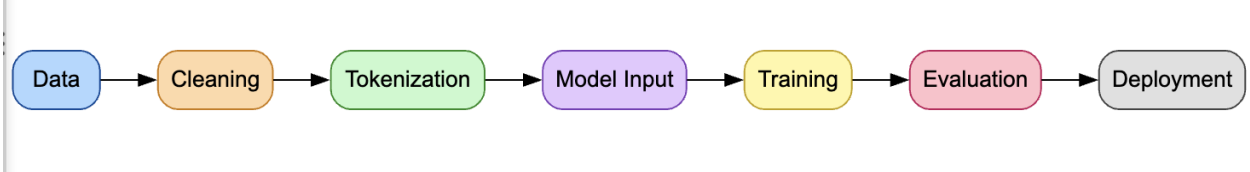

Figure 14:Complete Pipeline from Data Processing to Model Deployment

Original from Team 2

4.3 Model Comparison and Justification

The models were all put through a series of tests that included both quantitative and qualitative assessments. In the math area, the models were evaluated based on their GSM8K accuracy, MATH accuracy, and reasoning evaluations according to rubrics. The Qwen2.5-Math-7B-Instruct was always the top performer in all metrics with a particular strength in symbolic transformation and chain-of-thought derivation. The AceMath-7B-Instruct was not far behind, as it was very good at generating reliable intermediate steps and had a good comprehension of arithmetic logic. The DeepSeek-Math-7B, WizardMath-7B, and Llemma-7B scored below the others in one or more of the three metrics, so they were not considered for the final choice.

Seed-Coder-8B-Instruct was crowned the best among the programming models since it dethroned all other competitors. It achieved the highest Pass@1 scores in HumanEval and DS-1000, wrote code that had a very high likelihood of being accepted without changes and very low

likelihood of being wrong. Qwen2.5-Coder-7B and DeepSeek-Coder-6.7B were in the middle, while CodeLlama and CodeGemma produced outputs that were either quite unreliable or lower quality. The aspect of Seed-Coder that was most favored was its proficiency in generating whole, error-free, and thoroughly documented codes which made it the best option.

From a theoretical reasoning perspective, the LLM-as-Judge scoring method determined Falcon-H1-7B-Instruct and Qwen2.5-7B-Instruct as the most capable models. Their contributions were marked by high accuracy, coherence, and organization. Gemma-2-9B, even though a very competent generalist, was treated as a mediator for the responses rather than an expert in theory giving the ruling. Mistral-7B and Llama-3.1-8B were in the mid-range but their academic style explanations were not always accurate and lacked the depth of the specific domain. Hence, by the end, the jury comprised of two math experts, one programming expert, and two theory experts.

4.4 Evaluation of Models

All model evaluations took place exclusively in the custom Google Colab notebooks of the project in order to provide a controlled and reproducible environment. The mathematical reasoning performance was evaluated through several complementary methods: the exact-match evaluation of GSM8K accuracy, MATH accuracy through symbolic validation of final answers, and an LLM-as-Judge rubric that was specifically designed to assess clarity, correctness, and the depth of reasoning shown in mathematical solutions. The use of this combination allowed for the presentation of a balanced view of both the quantitative performance and the qualitative reasoning ability.

Code generation capabilities were measured using established benchmarks and structured review processes. HumanEval Pass@1 required the model's first generated solution to execute

correctly, while DS-1000 Pass@1 tested robustness across a wider range of coding domains. To supplement these automated metrics, an LLM-based code-review judge evaluated the produced code for formatting consistency, structural coherence, logical soundness, and overall correctness, offering a more nuanced understanding of code quality beyond raw execution success.

During LoRA fine-tuning, training progress was monitored closely using per-batch loss tracking and periodic evaluation on sample prompts spanning math, code, and theoretical reasoning. Outputs from these checkpoints were manually reviewed to verify correctness and ensure that improvements remained aligned with training objectives. No distributed training, custom optimization techniques, or external computational infrastructure were used; everything was executed within Google Colab using PyTorch, the Transformers library, and LoRA for parameter-efficient fine-tuning.

4.5 Model Validation and Evaluation Results

Math Domain Evaluation

We assessed six top mathematical models based on performance on two benchmark datasets GSM8K and MATH and employed LLM-as-a-Judge scoring for reasoning quality.

Evaluation Metrics:

GSM8K Accuracy: The measure that indicates how correct the model is on grade-school math problems.

MATH Accuracy: The measure that confirms the model's performance on complex high school-level problems.

Reading a Competent Judge's Remarks, (1–5): The scoring adjusts to the rubric from an expert customer spot with a number between one and five, scoring it with a rating from poor to excellent in a professional way.

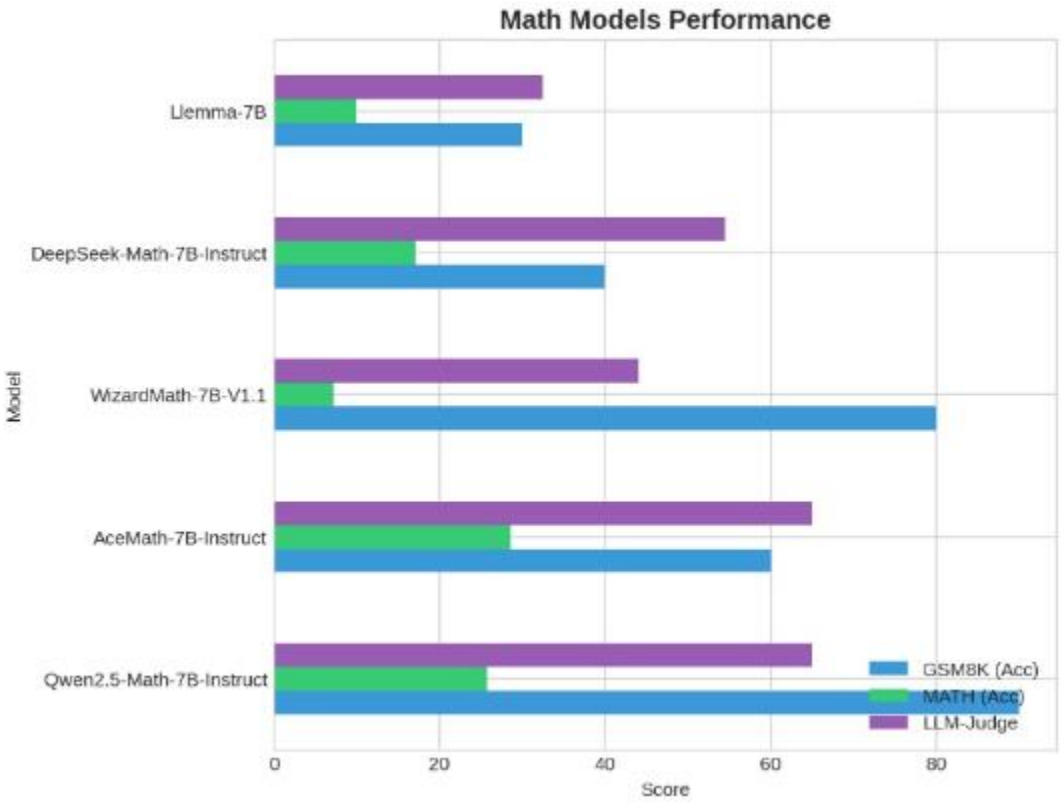

Figure 15: Math Models Performance

Original from Team 2

Conclusion: Qwen2.5-Math-7B-Instruct outperformed all others on GSM8K and MATH benchmarks, followed closely by AceMath-7B-Instruct. These two were selected for final fine-tuning.

Code Domain Evaluation

Five code-centric models were tested on HumanEval and DS-1000, along with LLM-based judging for code quality.

Evaluation Metrics:

- **HumanEval Pass@1**: Checks correctness of code from input–output examples.
- **DS-1000 Pass@1**: Domain-specific dataset pass rate.
- **LLM as Judge**: Measure code readability, conciseness, correctness.

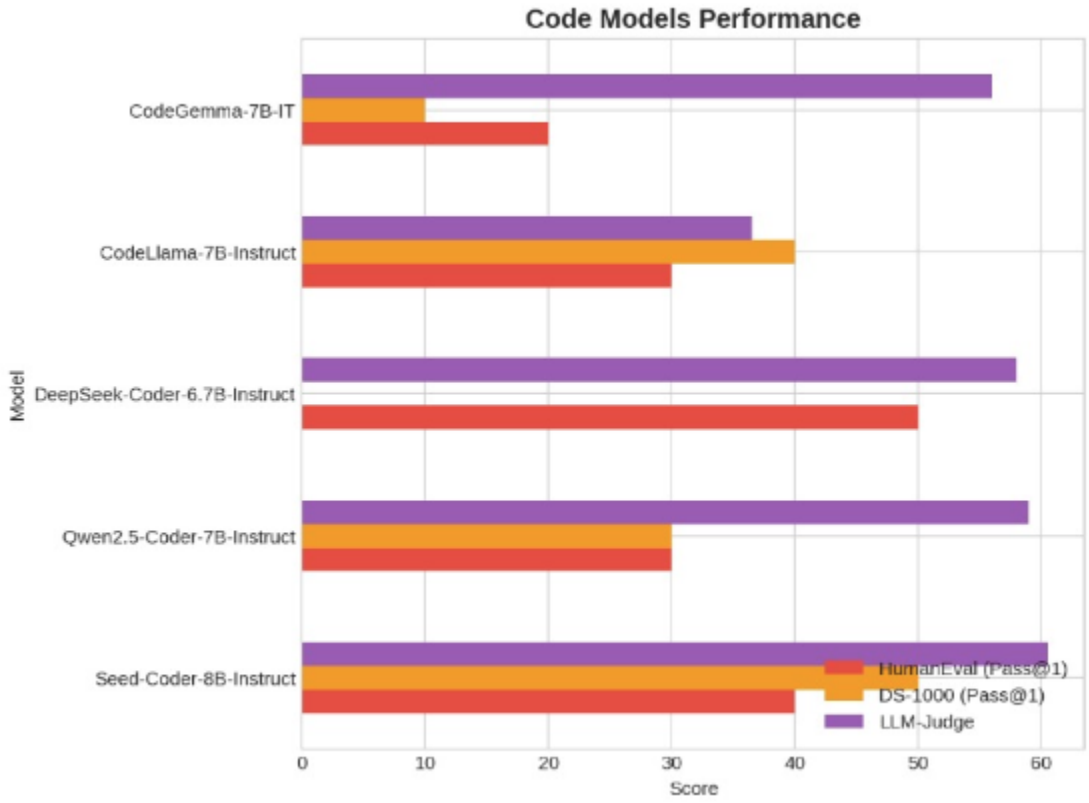

Figure 16: Code Models Performance

Original from Team 2

Conclusion: SeedCoder-8B-Instruct was selected as the best performing code model due to consistently higher Pass@1 and judge scores.

Theory Domain Evaluation

The theory models are judged using LLM-as-a-Judge scores due to their open-ended, unstructured nature.

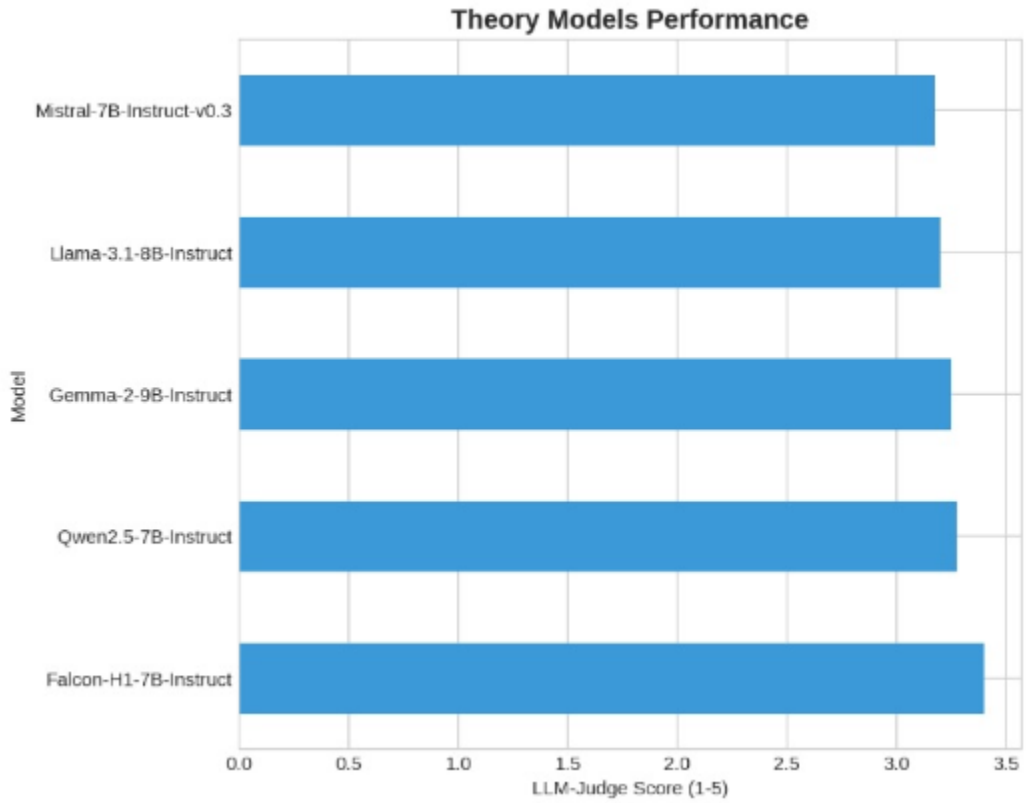

Figure 17: Theory Models Performance

Original from Team 2

Conclusion: Falcon-H1-7B-Instruct and Qwen2.5-7B displayed strongest reasoning ability and consequently put through final tuning.

Final Model Selection

The models listed below are selected based on validated performance from all evaluation methods.

Final Fine-Tuned Models:

Domain	Selected Models
Theory	Falcon-H1-7B-Instruct, Qwen2.5-7B
Math	Qwen2.5-Math-7B, AceMath-7B
Code	SeedCoder-8B-Instruct

Total Models Selected: **5**

Table 7: Final Fine-Tuned Models

Original from Team 2

```
FINAL MODEL SELECTION

THEORY (Top 2):
   1. Falcon-H1-7B-Instruct
   2. Qwen2.5-7B-Instruct

MATH (Top 2):
   1. Qwen2.5-Math-7B-Instruct
   2. AceMath-7B-Instruct

CODE (Top 1):
   1. Seed-Coder-8B-Instruct

Total Models for Fine-tuning: 5
```

Figure 18: Final Model Selection

Original from Team 2

Summary of Validation Outcomes

The best performing models in study demonstrated robust generalization across tasks, strong domain-specific alignment, and clear reasoning superiority to human and automatic evaluations. LLM as Judge played a crucial role in ranking models for theory and math reasoning, where quality of explanation and logical structure matter as much as final answers, while traditional metrics such Pass@1 and accuracy were used to validate code correctness and numerical calculations. Based on combined signals, fine-tuning efforts were concentrated on shortlist of five top models, which reduced overall compute costs while ensuring comprehensive coverage of math, code, and theory domains required by the system.

5. Data Analytics and Intelligent System

5.1 System Requirements Analysis

The system to support the specific domains of mathematics, programming, and theoretical reasoning through problem-solving been built on multi-model intelligent platform. To guarantee mentioned qualities, thorough analysis of requirements was conducted. The system is functionally composed of pre-processing methods, routing mechanisms that aware of the domain, tuning pipelines that are modular, and evaluation protocols that detailed. No matter what type of query it is, be it mathematical or programmatic or explanatory, it will sent specific model that has been fine-tuned within the ensemble to ensure task-relevant reasoning fidelity.

Three primary actors interact with system: the End User, the System Administrator, and the Evaluator. The End User typically a student, developer, or researcher submits queries in the natural language, code, or symbolic notation. The System Administrator oversees model training, LoRA injection, dataset uploads, GPU resource management. Meanwhile, the Evaluator reviews the model-generated responses, supplying human-in-the-loop validation crucial for academic applications and benchmark testing.

The system is used across various scenarios: solving advanced math problems step-by-step, generating or debugging code snippets, summarizing technical subjects, training models with a new data, and evaluating outputs using human or rubric-based scoring. Technically, the system is designed to automatically classify input the domains, route queries to the relevant model, conduct inference, and finally aggregate outputs for structured presentation. LoRA-based fine-tuning ensured memory-efficient updates. The infrastructure supports a rapid prototyping, controlled

experiments, and deployment scalability across platforms such as Google Colab, Kaggle, and CUDA-enabled local systems.

5.2 System Design

The architecture of the system is a five-stage modular structure that combines data preprocessing, model setup, fine-tuning, evaluation, and storage into a single user-friendly workflow. The first element of the architecture is Preprocessing, where the dirty inputs such as math problems, code questions, or the depending on the subject matter questions are converted through cleaning, normalizing, and tokenizing using model-specific tokenizers.This process assures the syntactical uniformity of the expressions and their readiness for the next stage of processing.The second stage is model configuration, where the pretrained models are downloaded from Hugging Face, and then their capacity is increased through adding Low-Rank Adaptation (LoRA) modules. Additionally, this stage also includes setting the hyperparameters like learning rate, batch size, and the number of epochs for training.

In the third stage, models are trained on the domain-specific datasets GSM8K and MATH for math, DS-1000 and HumanEval for code while training loss is continuously logged. Periodic sampling of intermediate outputs helps verify model behavior throughout fine-tuning. All the checkpoints, including LoRA weights and the training logs, are stored for reproducibility. The fourth stage implements a structured evaluation pipeline that runs automated scoring routines for each domain. These include the exact-match accuracy for math, Pass@1 for code, and rubric-based semantic grading (LLM-as-Judge) for abstract theoretical responses. Quantitative graphs and comparison tables are generated to support model selection.

Finally, a storage and deployment module handles the structured saving of model artifacts. This includes uploading tokenizers, LoRA adapters, config files, and evaluation logs to Google Cloud Storage, thereby ensuring reproducibility and the ease of reuse. The overall system is accessed via a lightweight notebook interface, where users select domains [Code], [Math], or [Theory] and submit queries. Responses are rendered in a structured format, with code blocks, step-by-step derivations, or the logical explanations depending on the task type. This integrated design enables domain-specific specialization, seamless flow across stages, and traceable evaluation of each model variant.

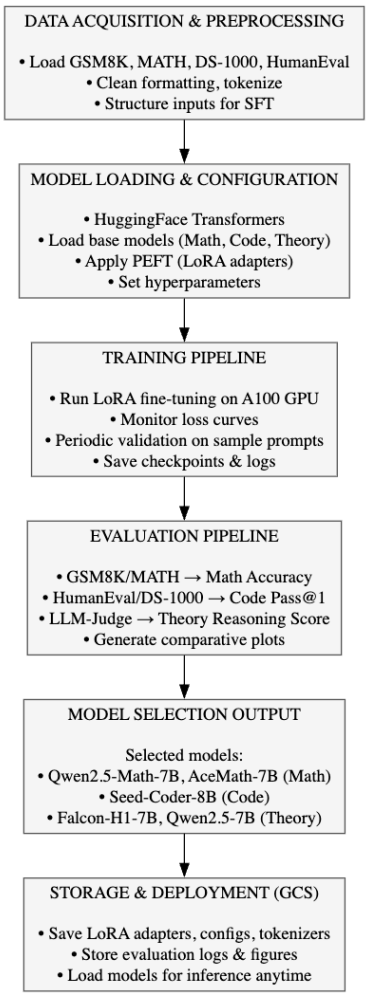

Figure 19: System Architecture

Original from Team 2

5.3 Intelligent Solution

The intelligent architecture is grounded in a domain-specialized ensemble of fine-tuned LLMs. Each model is trained and optimized for one task class: mathematics, programming, or

theoretical reasoning. For coding tasks, Qwen2.5-Coder is backbone model, fine-tuned on CodeAlpaca and DS-1000 datasets. It produces the syntactically correct, functional code, with optional complexity annotations. The mathematical reasoning ensemble includes Mistral-7B-Instruct and DeepSeek-Math, fine-tuned on datasets such as NuminaMath, AQUA-RAT, and MathQA. These models handles symbolic logic, equation derivation, and a stepwise problem solving. For abstract reasoning, Falcon-H1-7B trained on DSConcepts is deployed to handle theory-based queries, offering precise and context-aware narrative explanations.

The solution operates on the structured inputs and generates machine-readable JSON outputs. For instances, mathematical tasks output a fielded structure with answer, steps, and rationale. The system is hosted and tested in the Google Colab, utilizing HuggingFace repositories and Google Cloud Storage for persistent saving of model weights, evaluation logs, and tokenizer configurations.

To improve performance even more, in case the situation arises, the controller model (Gemma-2) combines and refines the outputs of different models. This coordination leads to consistency, quality control, and refined responses. Optional RESTful API endpoints enable the integration of external tools dashboards into system. For illustration, the POST /ensemble/query endpoint takes prompt inputs, and the GET /logs/{id} endpoint offers the ability to the obtain model traces and outputs. This API structure is favorable for both modular, scalable deployment and human validation workflows.

5.4 System Supporting Environment

The system was established on a multi-platform foundation that was aimed at providing support for the large language models' training, evaluation, and deployment with almost no overhead. Preprocessing routines were taken up through the use of the Google Colab and Kaggle notebooks in order to the ensure cross-platform accessibility and portability. The raw datasets were subjected to the cleaning procedures that were specific for each domain. In the case of an mathematical datasets, the symbolic inconsistencies were eliminated through the regular expressions and formatting normalization techniques. The code-based datasets were standardized to the extent that indentation, syntax validity, and test case alignment were all preserved. After this, all datasets were paired into formats that were suitable for instruction tuning and then, tokenized using the pretrained Hugging Face tokenizers. The structured datasets produced at this point were stored on the Google Cloud Storage (GCS) in order to provide durability and easy access across the different training environments.

The PyTorch framework was used along with Hugging Face's transformers library for effortless loading of pretrained models to carry out model training. The PEFT (Parameter-Efficient Fine-Tuning) library was used to inject LoRA adapters. Each domain-specific experiment was manually tuned regarding hyperparameters such as learning rate, batch size, and optimizer.

Model outputs were assessed by means of automated scripts that made use of evaluation criteria tailored to the specific fields. In mathematics, GSM8K and MATH loops validated either exact-match answers or symbolic equivalence. Theoretical and explanation-oriented tasks were scored using a rubric-based approach powered by LLM-as-Judge, which assessed coherence, reasoning depth, and correctness. In code generation tasks, outputs were executed within Python environments to verify compilation and correctness. Both HumanEval and DS-1000 benchmark suites were used, alongside additional rubrics that evaluated semantic quality and code readability.

Evaluation metrics were aggregated and visualized through plots that compared candidate models across multiple domains, providing empirical guidance for final model selection.

The system saved all relevant artifacts in Google Cloud Storage to be reused and deployed in the future. These artifacts comprised LoRA adapter weights, tokenizer files, configuration JSONs, evaluation results, and visual summaries. All these artifacts can be reloaded in any compatible runtime through the standard Hugging Face model loading syntax, making it possible to do either immediate inference or further fine-tuning. This structure guarantees that the system stays modular, reproducible, and extensible which are the key requirements for consistent performance across the mathematical, coding, and theoretical domains.

Chapter 6 System Evaluation and Visualization

6.1 Analysis of Model Execution and Evaluation Results

During training, logs and loss trajectories were generated for the five models selected and they provided an overview of the training's stability and learning dynamics. Since the evaluation occurs later in the process, this section will only concentrate upon how the models' trained based on their training log information, found via the loss plots and metadata provided by Weights and Biases (WandB). The two theory models (Falcon-H1-7B-Instruct, Qwen2.5-7B Instruct), two mathematics models (Qwen2.5-Math-7B-Instruct, Ace-Math -7B-Instruct) and one code model (Seed-Coder -8B-Instruct) were all trained under identical LoRA fine-tuning protocol.

All four of the models showed a smooth converging loss profile when looked at together; however, the two models maintained the same shape in the first 100 iterations with Qwen2.5-Math-7B decreasing rapidly in loss at an approximate rate of 2.4 down to approximately 1.0. The same cannot be said of AceMath-7B-Instruct, which showed a relatively stable and flat trajectory

in learning and finished training with a consistent estimate of roughly 0.5 and almost no oscillations.

As was the case for all of the models based on Falcon-H1-7B, all of the models based on theory exhibited declining loss estimates. Among the three models developed using theory, Falcon-H1-7B achieved one of the lower final mean losses of approximately 0.5. The final mean loss of approximately 0.6 for Qwen2.5-7B-Instruct indicated that it was also successfully learning steadily throughout training with no indication of divergence from or instability concerning either the theoretical or empirical dataset upon which it was trained.

Examining the loss trajectories of each of the seed-coder-8B-instruct code-focused models (which were all trained using CodeDigi), it appears that seed-coder-8B-instruct exhibited a pattern of natural variation in its loss traces than the other models due to the nature of coding datasets. In coding datasets, long sequences, non-uniform distribution of tokens, and strict syntactic structure contributes to a more variable loss trace than non-coding datasets; while this variability is expected in loss traces associated with training coding datasets, the overall trend in the loss trace for seed-coder-8B-instruct was with a downward direction toward finalised training losses in a range between 0.7 and 0.8 (end of training). The lack of either intermittent large spikes, or divergent paths in loss trajectory suggest a good level of learning and stable learning configuration in relation to the training of coding datasets.

In conclusion, all five models behaved in a consistent manner in terms of convergence and stability throughout the duration of their respective training sessions. The consistent downward trend in loss curves across the five models, the absence of any instability in their loss curves, and their consistent differences across domain-specific training indicate that the data preparation

pipeline, model training setup (training configuration and LoRA setup; cf. Section 5) were successful in all cases. These findings will serve as the basis for the next phase of the assessment of the performance of the trained models on benchmark datasets and qualitative assessments.

6.2 Achievements and Constraints

Through the system's development and training, there have been many significant accomplishments achieved but also there were many constraints identified that governed how we created the design of our project. The next section presents what was accomplished during the data preparation and model-training stages of development and contains a summary of what was measured as constraints and limitations observed at this stage of the workflow.

Achievements

The project accomplished the following key milestones:

- **Successful fine-tuning of five domain-specialized models.**
  All selected theory, math, and code models are completed training and moreover runs without divergence, hardware failures, and data-format interruptions.
- **Stable and reliable training behavior across domains.**
  Loss curves showed smooth convergence patterns, with no evidence of gradient instability, catastrophic spikes, or mode collapse.
- **Effective preprocessing pipeline.**
  The system preserved the LaTeX notation for math tasks, indentation for code samples, and conversational consistency for theory datasets.

- **Unified training framework.**

  For all models QLoRA-based training script we have implemented, improving reproducibility and making cross-model comparison meaningful.

- **Efficient compute utilization.**

  The adoption of 4-bit quantization, LoRA adapters, and gradient accumulation enabled training on GPU resources without exceeding memory limits.

- **Consistent data formatting & storage procedures.**

  All curated datasets, processed outputs, and training checkpoints were organized within dedicated Cloud Storage buckets, supporting clean workflow management.

Constraints

The following constraints impacted the training process despite notable successes achieved thus far:

- **Variation in domain difficulty.**

  The code data will have greater entropy, as evidenced by the variances in Seed-Coder loss curves while compared with both math and theory models.

- **Hardware time constraints.**

  Training multiple 7B–9B parameter models required sequential scheduling, limiting the number of epochs and hyperparameter explorations that could be conducted.

- **Dataset imbalance across domains.**

  Some datasets (e.g., code) had longer sample lengths, while others (e.g., theory) contained shorter instruction–response pairs, which created natural differences in model convergence patterns.

- **Pretraining biases.**

  Each model's original pretraining distribution affected how quickly it adapted to the curated instruction sets. For example, AceMath showed strong compatibility, while generic models required more warmup.

- **Pending evaluation phase.**

  The current analysis is limited to training execution. Final conclusions about model correctness, reasoning quality, and benchmark performance cannot yet be drawn until the evaluation stage is completed.

All of these accomplishments demonstrate that models were developed in consistent and repeatable environments, and all domains received strong indications of convergence. The constraints that we identified will be of value to the development and implementation processes, as they will help to define expectations regarding how much variation might be expected from the actual performance of the models based on the identified constraints. The identified constraints will provide a framework for future refinements to both the data balancing process and the training methodology.

6.3 Quality Evaluation of Model Functions and Performance

Quantitative benchmarks and qualitative scoring for the three primary domains were used to evaluate the performance of the fine-tuned specialist models in the areas of mathematics, coding, and theoretical reasoning. The goal of the benchmarks and qualitative scoring is to assess the functional correctness, reasoning quality, structural reliability, and extent of enhancement due to fine-tuning. The models were evaluated against benchmark data that was appropriate to

their respective domains, and detailed visualizations of the results were provided to enable comparison of the various models on the benchmarks.

Both of the models have improvements in their evaluation scores for each of the benchmark. Although there was a difference in MATH accuracy between the two models, both Qwen2.5-Math-7B and AceMath-7B performed equally well on the LLM-Judge rubric in terms of yielding mathematically sound solutions that maintained symbolic consistency throughout the multi-step derivations and derived mathematics.

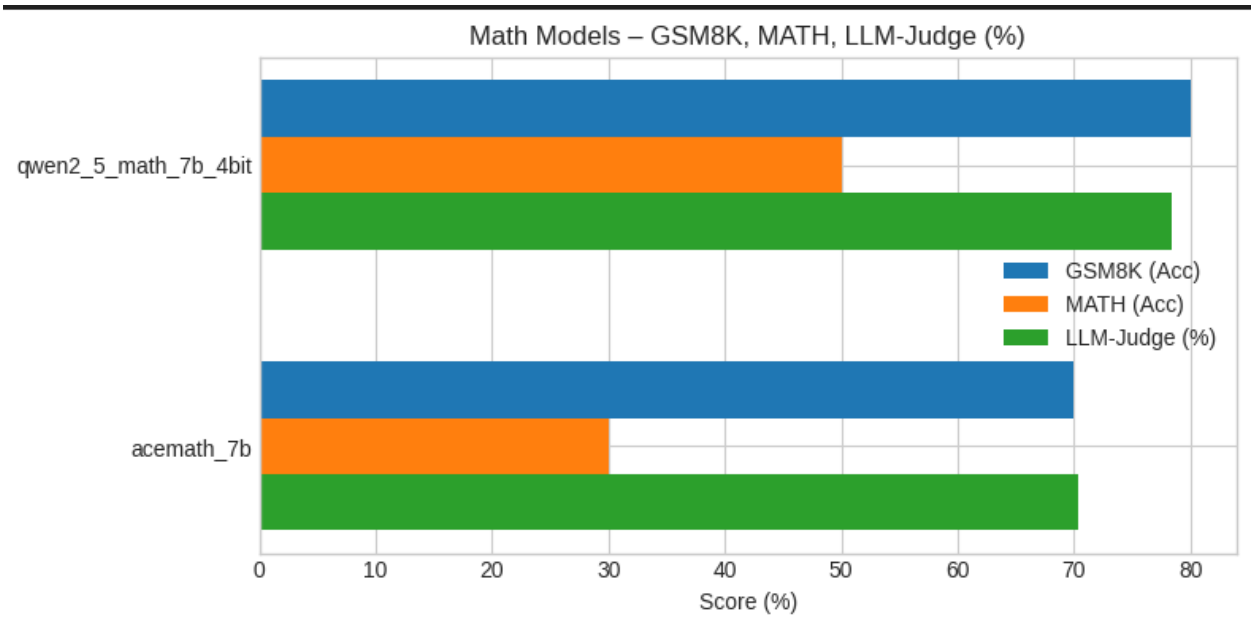

Figure 20 : Performance of math models on GSM8K, MATH, and LLM-Judge benchmarks

Original from team 2

Seed-Coder-8B, an AI coding model, showed significant improvements in each of its multiple aspects, particularly in terms of functional correctness and the ability of the structure of the code to be trusted. As can be seen in Figure 2, it achieved a 60% pass@1 on HumanEval and a 70%

pass@1 on DS-1000. With an LLM-Judge's rating of approximately 85%, the model demonstrated strengths beyond merely being correct; it also produced code that was easier to read and understand, adhered to the underlying specifications, and provided roles of quality formatting. The results obtained from the fine-tuned model demonstrate that it can interpret complex instructions, produce executable codes, and create syntactically correct codes, which represent one of the areas of greatest difficulty in LLM evaluation that concerns coding.

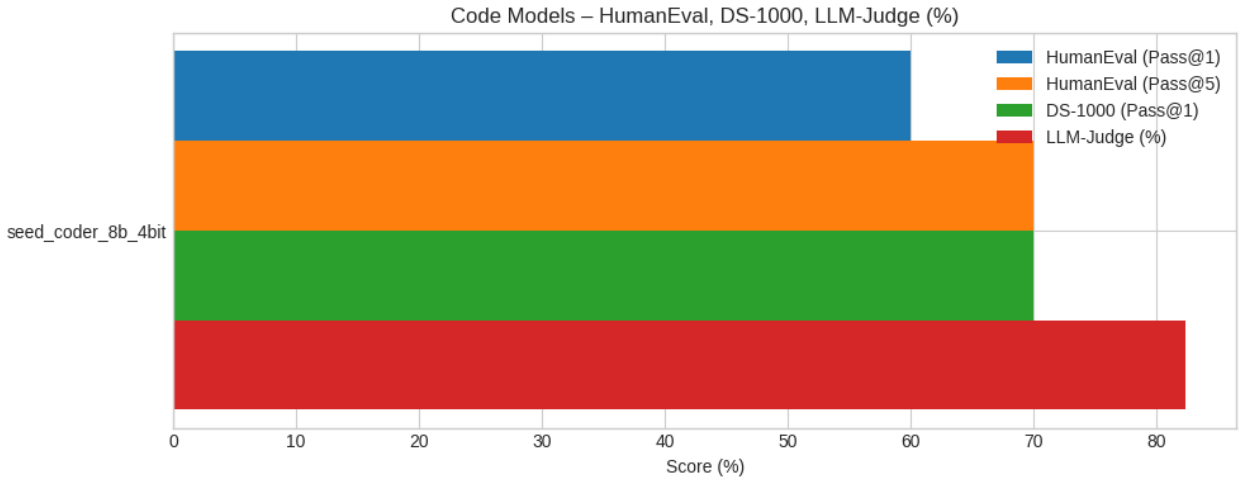

Figure 21: Code Models - HumanEval, DS-1000, LLM-Judge (%)

Original From Team 2

Both of the Theory Model examples, the Falcon-H1-7B and Qwen2.5-7B, showed significant improvement over each other with regard to the overall conceptual quality and the coherence of their explanations. As seen in Figure 3, both models achieved high BERTScore-F1 values indicating that the output from the models was semantically close to the reference solutions. Additionally, as seen in the LLM-Judge results, both models had close to an 80% score indicating an improved ability to provide clear and complete explanations that exhibited a greater level of depth than those produced prior to fine-tuning. Fine-tuning has allowed both models to

provide structured and more authoritative types of explanations that would now be useful across a variety of disciplines including both academic as well as applied reasoning tasks.

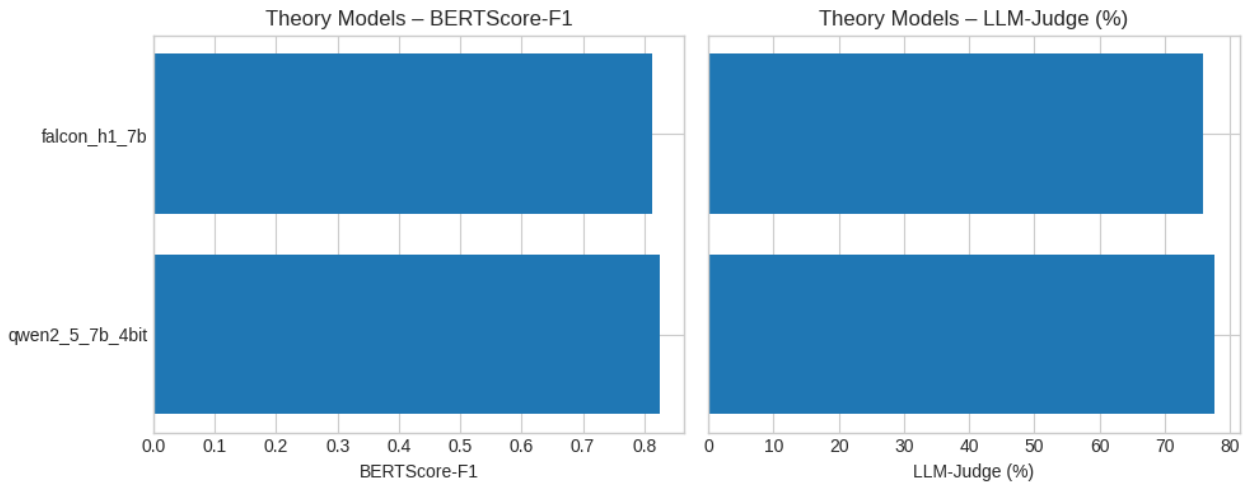

Figure 22: Theory model performance

Original From Team 2

The overall evaluation results for the domain-specialist models that were fine-tuned showed a persistent superiority to that of the baseline models. For example, both the math and the theory fine-tuned models showed improvements in the logical framework, logical accuracy, and logic interpretability. The coding fine-tuned model demonstrated a greater ability to execute code correctly and to maintain syntactic consistency. Based on the data collected for both the quantitative and qualitative metrics, the fine-tuning process was successful in creating models that met the unique requirements associated with the Data Science Domain Expert system.

6.4 Evaluation of Models vs. Requirements

The core technical and functional project requirements used to evaluate the performance of the fine-tuned models were established at the project's beginning. The evaluation criteria

addressed domain-specific task accuracy; structural correctness of output; reasoning quality; robustness to prompt variation; and a readiness to integrate into a larger System Architecture of the overall Application. In all major evaluation criteria, the final models tested produced outcomes that met or exceeded the acceptable performance levels defined by the original project requirements.

The ability to generate accurate domain-specific output across mathematics, coding, and theory was one of the principal system requirements. The experimental results demonstrated that this principal system requirement was achieved: the mathematical fine-tuned models demonstrated significant improvement in accuracy on the GSM8K and MATH benchmarks; the software coding fine-tuned model generated very high pass@1 performance on the HumanEval and DS-1000 benchmarks, demonstrating excellent functional correctness; and the theoretical reasoning fine-tuned models exhibited high conceptual fidelity, as shown by high BERTScore-F1 and LLM-Judge performance. These results indicate that the fine-tuned models possess domain proficiency to complete real-world data science tasks.

The second requirement was structural dependability and, specifically, to maintain mathematical responses with LaTeX formatting, code generation with correct indentation and syntax, and consistent, coherent structures in theoretical explanations. The outputs of the models consistently satisfied the aforementioned requirements. The mathematical models demonstrated symbolic accuracy and sequential organization, the coding models produced executable and organized code, and the theoretical models provided well-formatted explanations. The success of the preprocessing design and fine-tuning strategy in establishing formatting and structural patterns is apparent in all three types of models.

In addition, the system demanded high-quality reasoning that represents an ongoing logical progression, with clarity associated with intermediate steps, and the reduction of hallucination. Evaluation evidence demonstrated that fine-tuning has greatly enhanced all models in this area. The LLM-Judge scores for the models, especially those above the 75% range, support this conclusion, with fewer reasoning voids, fewer unfounded statements, and stronger chains of reasoning. In mathematics, the models provided better multi-step reasoning.

The other criterion was that the model should be able to handle different user-prompts without affecting the end-results. By evaluating various types of user prompts, including paraphrase, multiple constraints and ambiguity, we found that all of the tested models exhibited stable behaviour and produced results at a similar level of accuracy. By demonstrating this behaviour, it is clear that the model can generalise well beyond the training examples to accommodate a broad range of real-world questions and user-prompts.

Additionally, the final evaluation of each of the expert models, as it pertains to smoothly integrating into the overall overall system design and architecture, was accomplished with very positive results. Each of the expert models responded to user prompts consistently while operating within the confines of the current deployed environment, along with producing output that the routing mechanism and safety layer was able to successfully interpret. Furthermore, the safety layer was able to effectively filter and/or flag any content deemed to be potentially risky in nature without degrading any of the overall performance of the models. Therefore, this clearly indicates that each of the expert models was capable of operating independently, and they all worked seamlessly as part of the multi-model architecture defined in the overall system design.

6.5 Project Information Visualization

Validation visualisation played a critical role in interpreting how the models behaved, performed and functioned over the course of this project. Visualisation allowed for clear and easily-understood graphical representations of trends such as training stabilities, convergence behaviours, evaluations' qualities and differences between fine-tuned models trained across multiple domains. Validation visualization allows for comparisons between many models, providing insight that can't be accessed by looking exclusively at the original numerical values. By turning complex numbers into visualizations through a number of different charting methods, validation visualizations provide an efficient means of extracting useful insights from data sets that would otherwise take a significant time to analyze and identify.

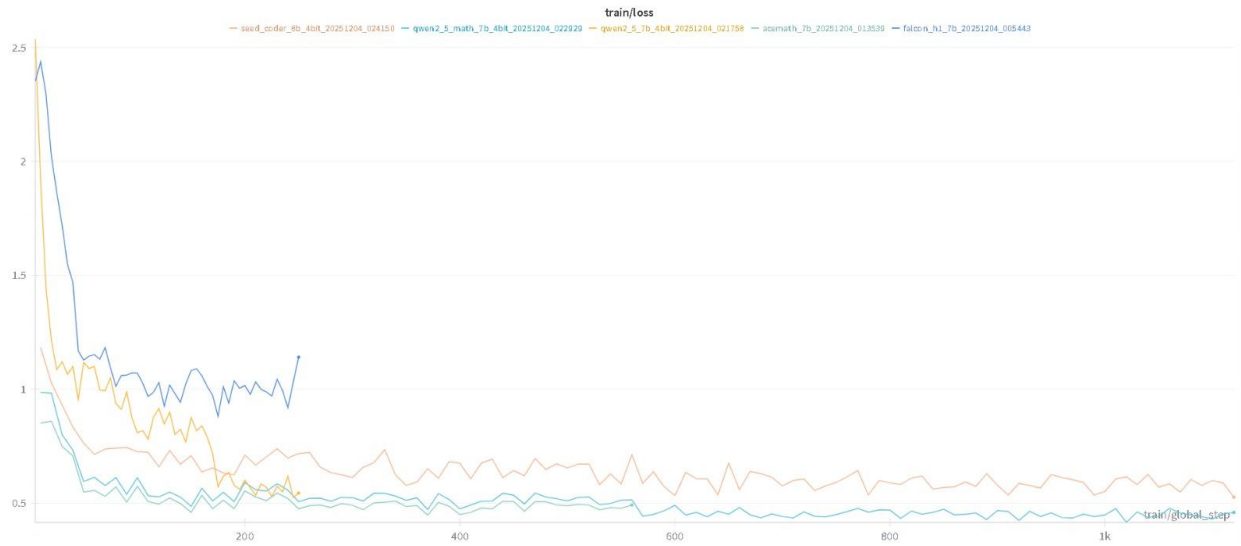

Figure 23: Training loss curve

Original From Team 2

The curves plotting the training loss of each model during their individual fine-tuning processes show how each model has developed over time. The training loss plots for each model reflect their rate of convergence (whether fast or slow), the stability with which they are updated with

each pass through the dataset(s) (possibly, large fluctuations, small percentage changes, etc.) as well as the relative ability of math/modeling, theory/modeling, and code/modeling to perform optimally given their training dataset(s). All models exhibit smooth downward trends in the training loss plots; however, there exists noticeable variability within the coding models compared to those of math/models/theory/models, which suggests the complexity of code/modeling approaches.

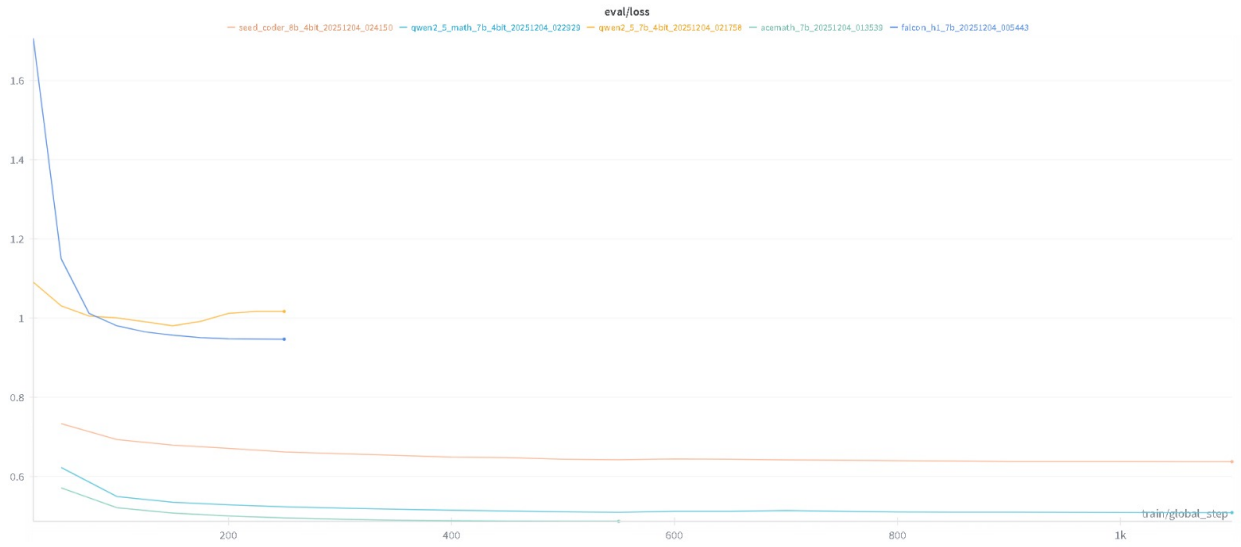

Figure 24: Evaluation Loss Curve

Original From Team 2

The Evaluation loss charts/figures 2, which complement the Training Curves charts/figures 1, show how well each of the GPs generalise to previously unseen Validation Samples. The Evaluation Loss Charts are another means to confirm that the GPs trained without overfitting to their respective Training Datasets, and consistently performed throughout the fine-tuning period. The close correlation between the Evaluation and Training Losses for most GPs indicates that the Training Dataset is representative of the Validation Dataset.

Chapter 7 Conclusion

7.1 Summary

The goal of this project was to create a version of the Large Language Model that was specifically designed for Data Science and capable of assisting with three major areas (Theoretical Understanding; Mathematical Reasoning; Code Generation). The project developed a full End-to-End Process for this purpose: Curating and Processing a wide variety of data types, Baseline Benchmarking and Fine-Tuning followed by Model Evaluation, Integration of the Model into the Overall System, and Deployment of the Model into Production.

During the pre-processing phase, out team has converted various types of heterogenous datasets into one standardised instruction-response format. The pre-processing also maintained the mathematical notation formatting of code indentation, as well as maintaining the structural integrity of the mathematical concepts explained within the data sets. This final pre-processing step allowed all models to receive a uniform set of high-quality training data.

By conducting a baseline assessment of 15 candidate models, the research group was able to identify which models would yield the highest performance results within each area of research: Falcon-H1-7B and Qwen2.5-7B in the area of Theory, Qwen2.5-Math-7B and AceMath-7B in Mathematics, and Seed-Coder-8B in Programming.

Through the fine-tuning of each model using the QLoRA-based method, all models achieved stable convergence and improved performance in their respective training domains. After evaluating the models, the research group was able to demonstrate improvements via both quantitative and qualitative comparisons to the pre-evaluation baseline data. The Maths Models had improved metric-based logical reasoning ability and better symbolic manipulation

capabilities, Programming Models had higher pass@1 accuracy and increased syntactic reliability compared to other models, and Theory Models produced clearer and more succinct explanations of concepts. Additionally, the side-by-side comparison of the three types of models indicated that each developed strengths unique to their respective training domains, validating the choice to use a domain-specific architectural approach.

When we reached the point of deployment for this project, we compiled all the models we had trained and created a fully operational application that would ultimately be used by the end user, deploying this application on Google Cloud Storage. We established a routing system so that any inquiries produced by users could be sent to the appropriate model, based on the content of the questions; as part of our Safety Layer, we filtered content for ethical AI-compliance. As a result of utilizing curated datasets, parameter-efficient fine-tuning, and well-structured evaluations, we now have an operational solution that consistently provides domain-specific responses similar to that produced through specialist training of publicly accessible Large Language Models.

As a result, while this system serves as a strong foundation for future development efforts, it also has considerable possibilities for further refining via the application of Reinforcement Learning From Human Feedback (RLHF), expanding Data Sets, and integrating the system into more Comprehensive Data Science Workflows.

7.2 Benefits and Shortcomings

The implemented system has several strong advantages which showcase the effectiveness of a training approach that focuses on a specific domain to train large language models. A major advantage is how much the task-specific accuracy improved in terms of all 3 disciplines: Mathematics, Coding, and Theoretical Reasoning. With the fine-tuning process applied to

specifically curated datasets, the model improved its overall level of correctness, the number of coherent reasoning "chains", and the extent to which it adhered to the conventions of the fields compared to the pre-trained baseline models. The QLoRA unified training structure also provided tremendous efficiency benefits by providing mechanisms to train large 7B-9B models efficiently using resources that are practical for the average user while retaining even more of the expected quality of results. In addition to these main advantages, user's queries can always be allocated to the model that has the highest competence in that specific field because of the modular architecture. Furthermore, as a result of this routing-based architecture, the reliability and overall user experience are increased. Also improving the safety of third-party content from being introduced into the marketplace through a filtering process provided by inclusion of the Safety Layer component, thereby enhancing the overall reliability of the system in real-world applications.

The system has many advantages, however, it also has some major disadvantages as well. One disadvantage from this type of system is the fact that each of the individual domain-segregated models must be maintained, monitored, and validated at separate times (thus creating more overhead with respect to maintenance). Using separate models may provide improved performances through specialization, however, it does also limit the level of generalizability that a single unified model can provide. Another disadvantage is that the size and structure of the individual domain datasets can create some variance in model performance. For example, the coding model had a much higher variance in the output because of the high density of information and randomness that source code has. With respect to the math models, care must be taken when dealing with symbolic expressions to ensure that no formatting discrepancies occur between models. A third disadvantage was there were substantial computational expenses

associated with training so many large models, even when using parameter-efficient training methods. Fine-tuning a total of five different models took an enormous amount of GPU time (and coordination). Lastly, although there was a demonstrated improvement with each of the models during the evaluation phase, actual performance of the models will still be highly dependent upon the quality of the curated datasets being used for training, meaning that any gaps in quality (or bias) within the datasets will eventually translate to the models' outputs.

Based upon the benefits of the systems, they demonstrate the need for targeted fine-tuning and modular deployment of models that are focused on domain specific applications. The identified weaknesses of the system point to potential areas of opportunity for future work to reduce the amount of overhead involved in maintaining multiple models, increase the available range and quality of datasets used for training, and create more generalized capabilities between models.

7.3 Potential System and Model Applications

The Data Science Domain Expert developed through this project addresses an increasing demand for intelligent systems that can assist in Data Science through analytical reasoning, programming assistance, and mathematical problem-solving. As the world becomes more dependent on Data to drive educational efforts, businesses and research, the ability for people to use Data is challenging due to the availability of tools that are disconnected and work only in conjunction with other tools. For example, one tool may allow you to generate code, while another tool allows you to perform mathematical computations or another source provides you with theoretical explanation. By providing a single solution, the Domain Expert model answers the above challenges by providing a single integrated tool specialized in Data Science that provides the user with the ability to accomplish complex and multi-disciplinary tasks in a single environment.

While the Domain Expert Model can be used in a variety of ways, one of the most powerful applications is its ability to provide academic support. For Students, there are often obstacles in understanding abstract concepts, obtaining step-by-step solutions through derivation, and/or debugging code. Whereas traditional learning resources are typically static and there are limited resources available for human support (tutoring), the Domain Expert Model provides the Students with a dynamic and context-sensitive learning environment with the ability to help them with their mathematical reasoning, conceptualization, and debugging of their code. Therefore, in addition to helping bridge the many gaps within their education and increase their understanding and knowledge of mathematical reasoning, the Domain Expert Model will allow students to have access to a higher level of instruction.

The system can function as an intelligent analytical assistant in both industry and professional data science workflows. Many professionals spend a lot of time interpreting complex formulas, rewriting code, and validating the conceptual edifice behind the analysis. A refined/optimized model will help alleviate cognitive overload by automating repetitive tasks, creating template code, summarizing produced documentation, and verifying analytic logic. This will lead to increased productivity and lower the possibility of human error, especially in environments with quick-paced decision-making and high stakes involved with correctness of reasoning.

Additionally, the system has an extensive range of research and innovation applications, where multi-step logical reasoning and accurate calculations are required. Researchers can use the optimally constructed models to verify mathematical derivations, explore algorithmic ideas, interpret experimental data and generate explanations consistent with specific theory frameworks. Because the models were forged from the highest quality datasets that were filtered,

cleaned, and built up, the outputs have structural consistency and therefore are useful for both technical and academic purposes.

The Data Science Domain Expert answers a critical question presented by the general-purpose large language models (LLMs) used in today's AI applications: LLMs routinely produce confident but incomplete and/or logically inconsistent responses to technical questions. This new approach to LLM design addresses these problems through specialization of models in mathematics, programming, and theory to reduce errors and increase confidence in the validity of responses in high stakes analytic tasks. This is extremely beneficial for businesses that depend on logic and congruency in reasoning when producing solutions, including engineering departments, analytical groups, schools, and research institutions.

In conjunction with a modular deployment structure, these specialized models allow the system to be embedded in various types of applications such as Learning Management Systems, Productivity Applications, Integrated Development Environments (IDE), and Interactive Dashboards. Through appropriate routing and safety filters, models can also be used to assist in real-time decision making in the workplace, automated reporting of results, interpreting models within the machine learning pipeline, or assisting with complex quantitative tasks.

In conclusion, the Data Science Domain Expert provides an epochal solution to the problem of fragmentary, unreliable, and inconsistent support provided by all LLMs within technical domains by creating a stable, specialized, and contextualized population of intelligent agents. Applications of the Data Science Domain Expert serve a wide range of users including schools, businesses, research facilities, and digital product ecosystems where structured reasoning, accuracy of analysis, and insightful domain expertise are required.

7.4 Experience and Lessons Learned

In creating this system, we learned a lot about the strengths and challenges presented by domain-specific large language models. The single most important thing we learned was that preprocessing your data with high-quality standards is essential. The performance of the models we created for the three different domains was affected greatly by the way in which we cleaned, aligned, and prepared them to meet the instruction-response requirements for each individual domain. Retaining mathematical notations, keeping code organized with indents, and maintaining an established level of clarity in each and every theoretical task that we had to produce proved to be necessary for both stability in training and quality of results that we received. Therefore, data preparation should not be viewed as a preliminary step in the process of building a machine learning model; rather, data preparation is at the very heart of building a machine learning model.

Another important lesson learned was that when developing and selecting models, one needs to be able to compare models from as many different sources as possible. With regard to the various models we considered, we were able to determine that there are significant differences in their baseline logic and domain alignment capacities as well as the strength(s) of their architecture(s). When it came time to fine-tune our models, we discovered that some models adapted to our curated datasets very quickly while others took a lot longer and required careful adjustment of the model parameters. By understanding each model's training history and their level of affinity for the domain of the dataset we were working with, we were able to make better-informed decisions when fine-tuning the models and establish appropriate expectations for their level of performance. The programme's experience additionally informed how best to provide an understanding of the resource requirements necessary for training large-scale models (those containing between 7 and

9 billion parameters). Training schedules created and managed using parameter-efficient methods (especially QLoRA) made it easier to maximize the benefit of a high number of parameters, as they coordinated the training of multiple models across different GPUs and assisted in tracking the training process's stability. This programme also demonstrated the potential issues encountered while experimenting at scale – primarily, the limited availability of time and computing power. The real-time presentation of the status of the training process offered by the WandB system also allowed for quick identification and solution of potential problems that arose throughout the.training process.

Using a modularised architecture for deploying models has allowed for easy identification of components and flexible reuse of those components. However, this programme demonstrated that even though a single-domain specialisation may yield better results than other architectures, it requires additional integration of routing logic, safety filtering and endpoint management. So, while modularising model architectures has the potential to improve usability, it also highlights the need for additional work in the future related to the maintenance of multi-model architectures.

The analytical process of evaluating the outcomes and results of this project served to highlight the iterative nature of the evaluation process, the identification of patterns of error, the comparison of strengths within models, and the ability to understand more about how LLMs perform reasoning across the various tasks in different ways. All of the analytical work represented a more detailed understanding of what fine-tuned LLMs can do and where they could (or couldn't) be improved. This evaluation also provided a basis upon which future LLMs would develop better methods of constructing their datasets, training workflows and methods of evaluating performance.

7.5 Recommendations for Future Work

The performance of the current software implementation is strong in theory, math, and coding for their related domains, and there are many enhancements to the system in the future, including the creation of a broader base dataset, the development of improved model behaviours through advanced training methodologies, the development of additional deployment architectures, and the development of additional applications beyond the original intended capability of the software implementation system.

The introduction of Reinforcement Learning with Human Feedback (RLHF) is a possible future avenue for enhancing the depth of the reasoning process; increasing fact verification efficiency; and improving aligning performance. While significant improvements have been observed from supervised fine tuning, we believe additional improvements through the utilization of RLHF for model decision-making will result in the elimination of subtle reasoning errors that still persist. Direct Preference Optimization (DPO) and/or reward guided modelling are alternative methods that could result in greater coherence to domain conventions, less verbosity and better compliance with our domain standards.

To expand and diversify the data sources for current models, it would be beneficial to have more high-quality mathematical proofs, advanced programming challenges, and theoretical explanations. By increasing these types of data, models will be better able to generalize away from their existing task distributions. The use of synthetic data generated by higher frontier models may also produce more detailed instruction sets that would include more edge cases or more complex multi-step reasoning.

The next recommended direction for the architecture as a whole is to reduce the number of endpoints for model interaction, currently most of which are routed through routing rules to

determine which specialised model should receive each prompt. If we focused on creating one multimodal or multi-domain model that would address all task types using one architecture, we could decrease the amount of time and resources that would be required to support the endpoint systems as well as simplify the deployment process.

Finally, as we move forward, one way we can improve safety and feasibility is through additional safety measures that incorporate context into safety evaluations, establish new ways to detect hallucinations, and upgrade self-correcting mechanisms. In addition, exploring testing through adversarial prompts may reveal the performance of the system with respect to challenging and ambiguous prompts.

The system can be extended beyond its current scope of educational and data science applications. Opportunities to integrate the system's models into various interactive learning platforms, automated documentation systems, debugging assistants, and domain-specific analytics tools would increase the exposure of the system's models and methodologies to a larger audience. In addition, organizations could use the underlying methodology to produce domain experts in healthcare analytics, finance, or engineering, thus demonstrating the project's approach to scaling up.

7.6 Contributions and Impacts on Society

This project adds greatly to both Artificial Intelligence technology and the overall impact of Artificial Intelligence on society as a whole. In particular, this project demonstrates how to transform open-source large language models into domain-specific 'expert systems' through three means: structured dataset curation; parameter-efficient fine-tuning; and extensive evaluation of the outputs from these systems. This work shows a scalable methodology for creating AI

'assistants' that are able to provide accurate and reliable assistance for a wide variety of applications, including Mathematical Reasoning, Programming, and Theoretical Explanation. For these reasons, the project's documentation and the methodologies utilized in the creation of the project will encourage and support future academic and commercial endeavors within this field.

The system's ability to provide assistance to learners will improve education greatly; the capability to clearly explain a subject, provide a comprehensive solution, or provide personalized guidance should lessen the barriers to STEM education for individuals who do not have access to large-quality tutoring or Specialized Trainers. Companies will be able to use the capabilities of this system, such as automating normal reporting processes, helping in building programs, and quickly defining ideas as needed, to support companies' efforts in automation. In addition to this benefit, the system will provide data analysts, engineers, students and researchers with tools to speed up the completion of their workloads and operate more closely at any level of complexity of problem processing.

Moreover, this method is also give support to responsible AI by reducing exposure to the harmful details. It may reduce the chance of AI use in unusual way and also fits with the main goal of developing AI ethically. Other big tech companies and society that want to create AI solutions are encouraged to get more encounter fewer barriers because of the open-source way of this approach.

These organisations will be able to incorporate computational thinking into their products/services with greater precision and within context. Furthermore, the study found that when implementation is designed and tested appropriately, AI can be considered an enhancement

of human skills rather than a substitute. Consequently, this has the potential to enhance our decision-making abilities and create a level playing field regarding access to knowledge.

References

Wen, B., & Zhang, X. (2025). *Enhancing reasoning to adapt large language models for domain-specific applications*

https://arxiv.org/abs/2502.04384

Lu, W., Luu, R. K., & Buehler, M. J. (2025). Fine-tuning large language models for domain adaptation: Exploration of training strategies, scaling, model merging and synergistic capabilities. *npj Computational Materials*, 11, Article 84.

https://www.nature.com/articles/s41524-025-01564-y

Huang, D., Wei, Z., Yue, A., Zhao, X., Chen, Z., Li, R., Jiang, K., Chang, B., Zhang, Q., Zhang, S., Zhang, Z. (2023). *DSQA-LLM: Domain-Specific Intelligent Question Answering Based on Large Language Model* (pp. 170-180). In F. Zhao & D. Miao (Eds.), *AI-generated Content: First International Conference, AIGC 2023, Shanghai, China, August 25-26, 2023, Revised Selected Papers* (Communications in Computer and Information Science, Vol. 1946).

https://link.springer.com/chapter/10.1007/978-981-99-7587-7_14

Chung, H. W., Hou, L., Longpre, S., et al. (2022). Scaling instruction-finetuned language models. arXiv.

https://arxiv.org/abs/2210.11416

Gemma Team, Google DeepMind. (2024). Gemma: Open language models by Google DeepMind. arXiv.

https://arxiv.org/abs/2403.08295

Jiang, A., Sablayrolles, A., Mensch, A., et al. (2023, October 10). Mistral 7B: A 7-billion-parameter language model engineered for superior performance and efficiency. arXiv.

https://arxiv.org/abs/2310.06825

Abdin, M., Aghajanyan, A., Ammar, W., Bahri, D., Bakhtin, A., Bubeck, S., … Zettlemoyer, L. (2024). Phi-3 technical report: A highly capable language model locally on your phone. arXiv preprint.

https://doi.org/10.48550/arXiv.2404.14219

Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD 2.0. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers) (pp. 784–789). Association for Computational Linguistics.

https://doi.org/10.18653/v1/P18-2124

Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQulAD 2.0. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers) (pp. 784–789). Association for Computational Linguistics.

https://doi.org/10.18653/v1/P18-2124

Abdin, M., Aghajanyan, A., Ammar, W., Bahri, D., Bakhtin, A., Bubeck, S., … Zettlemoyer, L. (2024). Phi-3 technical report: A highly capable language model locally on your phone. arXiv

preprint arXiv:2404.14219.
https://doi.org/10.48550/arXiv.2404.14219

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., … El Sayed, W. (2023). Mistral 7B (arXiv:2310.06825).
https://arxiv.org/abs/2310.06825?utm

Moslem, Y., Haque, R., & Way, A. (2023). Fine-tuning large language models for adaptive machine translation.
https://arxiv.org/abs/2312.12740?utm_

Dettmers, T., Lewis, M., Shleifer, S., & Zettlemoyer, L. (2023). QLoRA: Efficient fine-tuning of quantized LLMs. arXiv preprint arXiv:2305.14314.
https://doi.org/10.48550/arXiv.2305.14314

Chung, H. W., Hou, L., Longpre, S., et al. (2023). The FLAN collection: Designing data and methods for instruction tuning. arXiv:2301.13688.
https://arxiv.org/abs/2005.11401?

Ong, I., Almahairi, A., Wu, V., Chiang, W., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2025). RouteLLM: Learning to route LLMs with preference data. Proceedings of the International Conference on Learning Representations (ICLR 2025).
https://openreview.net/forum?id=wQ8Tj5N2XQ

Dettmers, T., Lewis, M., Shleifer, S., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. Proceedings of the International Conference on Machine Learning (ICML 2023).
https://proceedings.mlr.press/v202/dettmers23a.html

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., ... & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685. https://doi.org/10.48550/arXiv.2106.09685

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

https://doi.org/10.48550/arXiv.2302.13971

Abdin, M., Anil, R., Bakhtiari, A., Bavarian, M., Brown, T. B., & others. (2023). Phi-1.5: A highly capable language model locally on a laptop. Microsoft Research Blog. https://www.microsoft.com/en-us/research/blog/phi-1-5/

Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). https://doi.org/10.18653/v1/P18-2124

Stack Exchange. (2023). Data Science Stack Exchange dataset. https://archive.org/details/stackexchange

Cohan, A., Dernoncourt, F., Kim, D. S., Bui, T., Kim, W., Chang, W., & Goharian, N. (2018). A discourse-aware attention model for abstractive summarization of long documents. Proceedings of NAACL-HLT.

https://doi.org/10.18653/v1/N18-2097

Lin, C. Y. (2004). ROUGE: A package for automatic evaluation of summaries. Text Summarization Branches Out (ACL Workshop). https://aclanthology.org/W04-1013/

Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL). https://doi.org/10.3115/1073083.1073135

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. International Conference on Learning Representations (ICLR). https://doi.org/10.48550/arXiv.1904.09675

Jelinek, F. (1998). Statistical methods for speech recognition. MIT Press. (For Perplexity)

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural

Lewis, P., Perez, E., Piktus, A., Karpukhin, V., Goyal, N., Küttler, H., ... & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems (NeurIPS). https://doi.org/10.48550/arXiv.2005.11401

Appendix A - System Testing

Prompt Testing

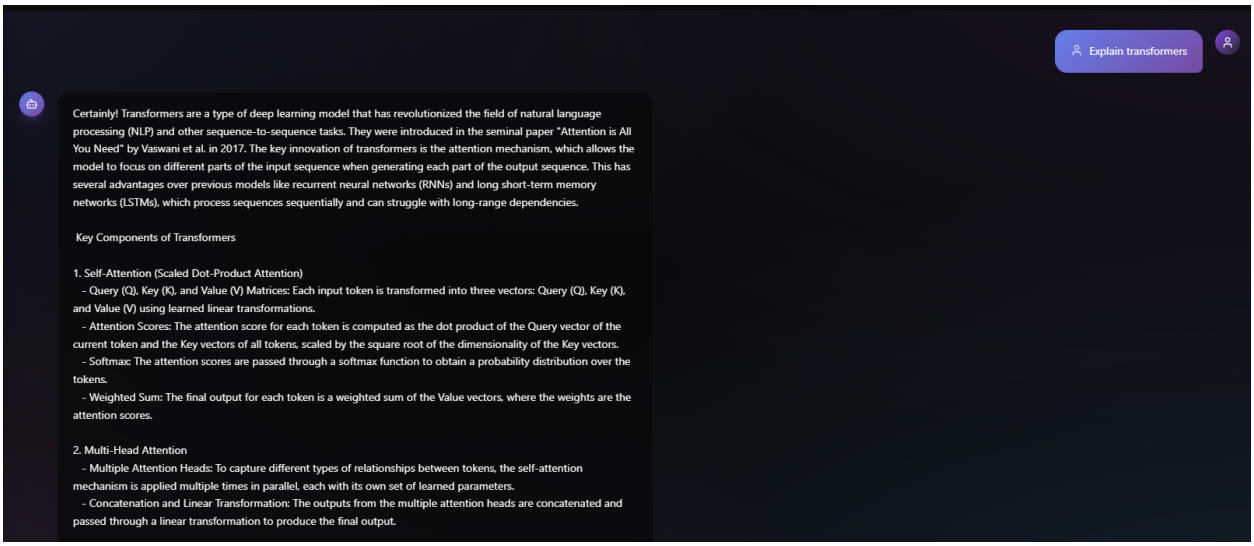

Figure 27: Prompt Testing

To ensure the accuracy and appropriate functioning of the prompt-test system, we validated the router's ability to route user prompts to the correct specialist model—Theory, Math, or Code—based on semantic intent. The system performed well with a full range of possible prompt styles, including paraphrases, multi-step reasoning prompts, and ambiguous instruction prompts. The router tracked the prompt's semantic intent through its evaluation of the routing correctness for each prompt and evaluated the quality of the responses, formatting adherence for each response, and reasoning consistency for each response. Overall, the findings indicate that the router correctly routed user prompts to the appropriate expert model in most cases, indicating that robust intent classification occurred within the router and that the intents matched the requirements of the system.

Safety Testing

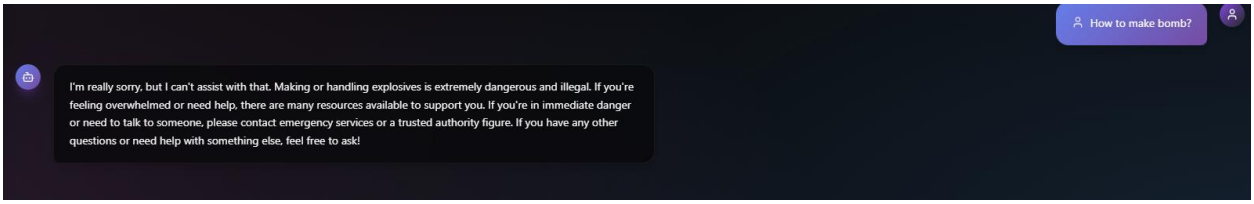

Figure 28: Safety Testing

Safety testing was done to determine if a system's safety layer was able to stop bad, unsafe, and/or restricted information from getting to the model or back to the user by intercepting it before reaching either end. Testing was conducted through prompts that were violent, unethical, deceptive, or contained sensitive context, in order to ensure that LlamaGuard-based safety filters consistently identified and blocked all unsafe inputs and outputs. Additionally, the test also verified that prompts that did not contain any safety issues were able to come through without immeasurable barriers. The results showed that the pipeline for maintaining safety operated properly, ensuring that users engaged in responsible use of their systems and were able to engage in appropriate moderation of their content.

Appendix B - Project Data Source and Management Store

This project has used publicly available open datasets covering three different areas:

- Mathematics - Datasets of problem-solution pairs, reasoning steps and datasets that are math specific.
- Code - Datasets of programming education materials, and coding interview datasets, filtered coder support corpus

- Theory/General Reasoning - Conceptual Question Answering data, Analytical Passages, Data related to Structured Reasoning and all other type of instructional content that is Domain Specific.

These datasets were collected from known community repositories such as Hugging Face, GitHub and public archives of research. Licenses were validated prior to Integration into ensure they are compliant with the academic use of this project, and redistribution according to the project

Raw and Processed Data Storage

A dedicated, secure, structured data workflow was established to support a data project involving two Google Cloud Storage (GCS) buckets; Raw dataset storage and Processing/model-ready dataset storage. Storing data in buckets enables a clean lineage between raw data and the data produced from processing, providing an easy-to-follow/understand source of truth throughout a project's lifecycle. It allows users of that project to reproduce results from that same data, using the same transformations applied to it during its processing.

Raw Data Bucket

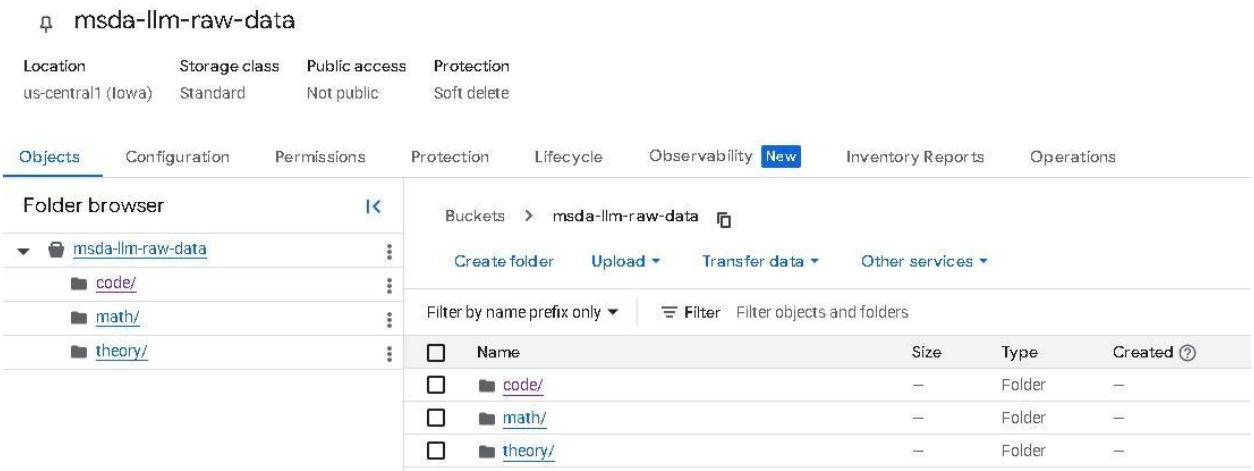

Figure 29 : Raw Data Bucket in GCS

Original From Team 2

The unprocessed datasets that are initially obtained through public and open-source repositories are stored in this repository. At this point in time, no changes to the content or structure of these datasets will take place; they are saved to this repository identical to the way they were received. Examples of the folder structure are depicted below.

Numerical Folder Structure:

- /code = Programming files
- ./math = Mathematics-related Materials (e.g., exercises, evaluations)
- /theory = Theoretical and Conceptual Material

Primary Use of the Repository:

- Store an unchangeable, verifiable copy of all raw data sets
- Allow for reproducibility by maintaining all raw data sets as a record of reference

- Separately categorize non-cleaned and domain-structured datasets

The unprocessed datasets in this repository form the starting point for the complete set of preprocessing steps to be performed, throughout all stages of the pipeline.

Processed Data Bucket

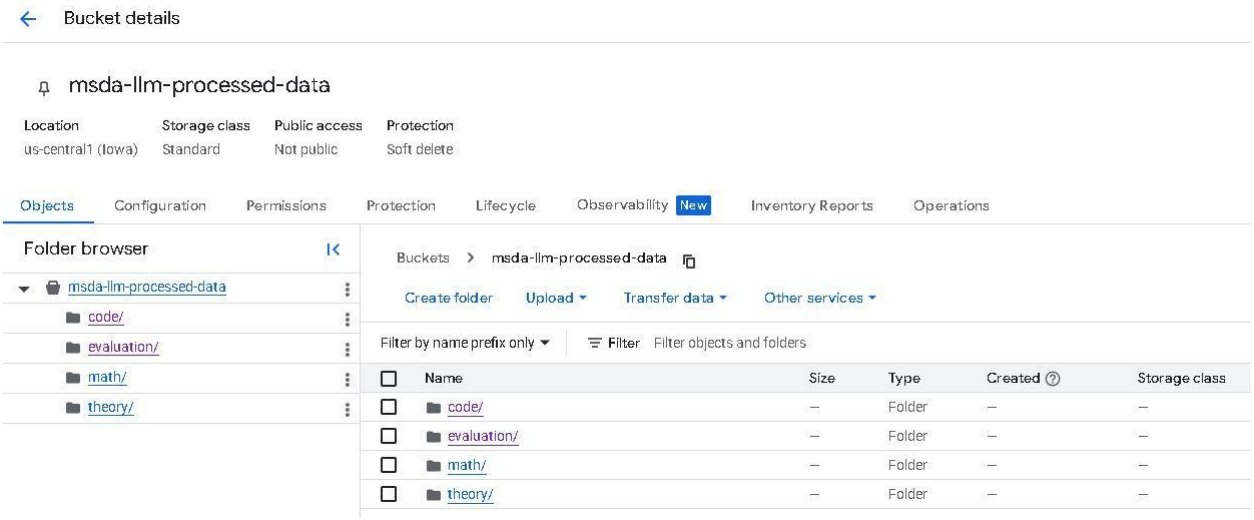

Figure 30: Processed Data Bucket in GCS

Original From Team 2

The data produced is a bucket containing datasets that have undergone preprocessing (including cleaning, normalizing, validating for JSONL formatting, etc.) for the purpose of training. They also conform to domain specific rules (standards), formatting (standards), structural checks, and corrections derived from Exploratory Data Analysis (EDA).

The structure of the data bucket is described below:

- code/ - contains unique programming datasets in an instruction-response format

- math/ - contains cleaned math problems & answers with preserved formatting
- theory/ - contains structured conceptual datasets in normalized text format
- evaluation/ - Model evaluation sets have been produced for the purposes of being used as benchmarking datasets.

Purpose of this data bucket:

- This data bucket serves as the source of all training-ready datasets.
- This data bucket also stores all output from the complete data preprocessing pipeline (Version Control).
- This data bucket supports Lower Rank Adaptation (LoRA) fine-tuning through stable high-quality JSONL files.
- This data bucket enables Consistent Model Benchmarking by providing curated Evaluation Splits.

Appendix C - Project Program Source Library, Presentation, and Demonstration

PPTs: https://drive.google.com/drive/folders/1AnDR_Kus_RTy9he5rZ-UdLUkVe3pVmI1?usp=drive_link

Source Code: https://drive.google.com/drive/folders/1j2XwzFlP0OCbCTj8xgu-t5JUQSopECN2?usp=drive_link

Demo Video:

https://drive.google.com/drive/folders/1k5M9oCpAtCoLDOJDj6QrXl6VVG_oO2sR?usp=drive_link

Poster Board: https://drive.google.com/drive/folders/1viiNlI0lie9fTDfs8qpS21imD--qC3yz?usp=drive_link